

APLICAȚII ALE MICROCONTROLERELOR
SISTEM DE PROCESARE DIGITALĂ A
SEMNALELOR CU COMANDĂ ARM ȘI EXECUȚIE
DSP (AGC, ALE, MF)
ECHIPA NR. 15

Studenti:

Anton George

Mazilu Riana Florentina

Perjaru Adrian

Tudorache Horațiu

Profesor coordonator: Zoican Sorin

1. Cerință și Descrierea Sistemului:

Să se realizeze arhitectura unui sistem format din două subsisteme (ARM și DSP), având rolul de a procesa semnale într-un sistem de comunicații digitale. Sistemul va respecta specificațiile din figura 1.

Microcontrolerul ARM (STM32F103CBT6) gestionează intrările utilizatorului și afișarea stării sistemului, în timp ce microcontrolerul DSP (ADSP2181) efectuează procesarea efectivă a semnalelor.

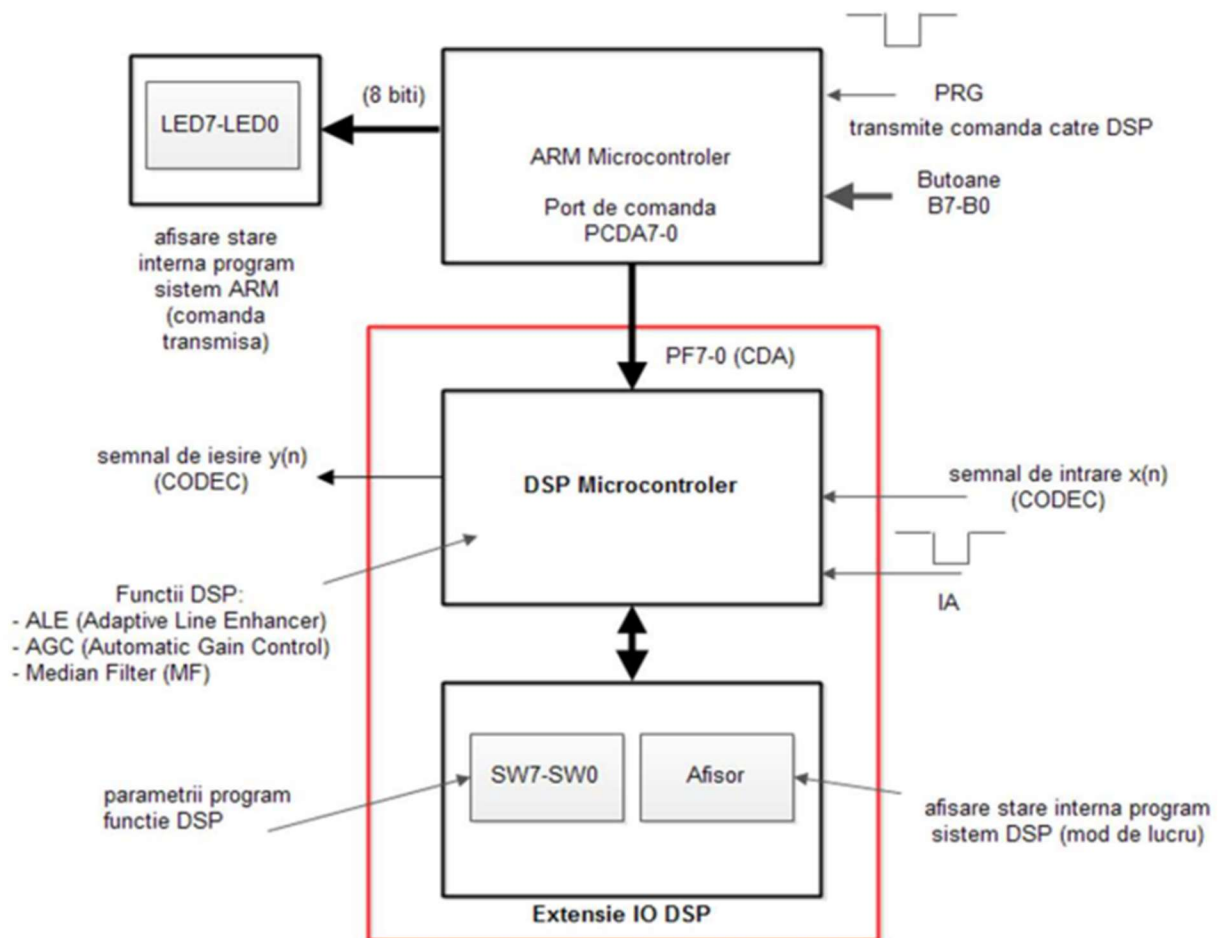


figura 1 – schema bloc a sistemului

Microcontrolerul ARM (STM32F103CBTx): Primește comenzi de la utilizator folosind butoanele B0-B7, care construiesc un cuvânt de comandă pe 8 biți, afișându-l pe LED-urile LED0-LED7 și transmițându-l către DSP prin intermediul portului de comandă PCDA7-0.

Microcontrolerul DSP (ADSP-2181 EZ-Kit Lite): Primește comanda oferită de ARM prin intermediul porturilor PF7-0, o validează și extrage parametrii (prezenți în structura cuvântului de comandă – funcția CDA, ID-ul parametrului, canalul), astfel încât să facă selecția funcției de procesare corespunzătoare. De asemenea, primește semnalul audio de intrare (de forma $x(n)$) de la CODEC-ul AD1847 și generează semnalul de ieșire (de forma $y(n)$).

CODEC-ul AD1847 SoundPort: Un convertor ADC/DAC pe 16 biți care are rolul de a digitaliza semnalul audio oferit la intrare și de a reconstrui semnalul la ieșire. Acesta comunică cu DSP-ul prin portul serial SPORT0.

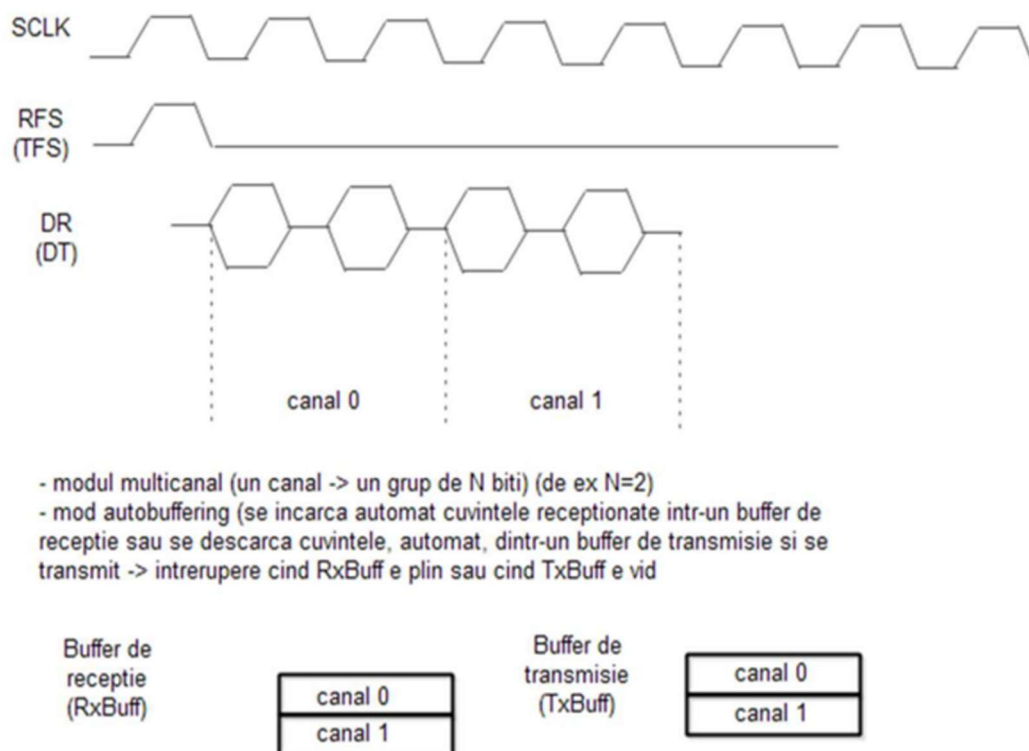


figura 2 – conectarea DSP prin portul serial SPORT0

EXTENSIA IO DSP: Aceasta include comutatoarele SW7-0 pentru configurarea manuală a parametrilor și un afișor folosit pentru a monitoriza stările interne ale DSP-ului.

2. Descrierea funcționalității porturilor de intrare și ieșire:

A) ARM

Microcontrolerul ARM acționează ca element de decizie și interfață cu utilizatorul, având rolul de a configura și coordona activitatea procesorului de semnal (DSP).

Intrările sistemului ARM:

Configurarea funcționării se realizează prin două mecanisme digitale:

- **butonul PRG (Trigger Comandă)** – este un *buton fără reținere* care, la apăsare, generează evenimentul de transmitere a pachetului de control către DSP
- **grupul de butoane B7-0 (Parametrii de Stare)** – acesta definește *cuvântul de control* care va fi trimis către DSP

Structura Cuvântului de Control (Magistrala PCDA7-0):

Conform schemei de soluție propuse, configurația biților este următoarea:

BIT	SEMNIFICAȚIE	DETALII
D7	1	Bit de validare/sincronizare.
D6	1	Bit de validare/sincronizare.
D5-D4	CDA1-CDA0	Selector pentru algoritmul DSP: ALE, AGC sau MF.
D3	CANAL	Selector pentru canalul de date.
D2-D0	ID2-ID0	Identificatorul DSP-ului țintă (permite adresarea individuală într-un sistem cu mai multe DSP-uri).

ieșirile sistemului ARM (Magistrala PCDA7-0):

ARM-ul comunică starea sistemului prin două căi principale:

- **interfața locală (LED7-0)** – aceste LED-uri sunt conectate direct la ARM și reflectă *starea internă* a programului. Ele confirmă vizual utilizatorului că pachetul de comandă a fost procesat și transmis corect
- **magistrala de comandă (PCDA7-0)** – reprezintă *portul de ieșire pe 8 biți* prin care ARM-ul transmite instrucțiunile către intrarea DSP-ului, PF7-0 (CDA)

B) DSP

Gestionarea Parametrilor din Switch-uri (SW7-SW0):

Logica Semnalului de Întrerupere (IA – Interrupt Acknowledge):

DSP-ul nu rulează codul la întâmplare, el este sincronizat cu CODEC-ul audio prin semnalul IA.

1. **Eșantionare ($x(n)$):** CODEC-ul transformă sunetul analogic în digital și ridică semnalul IA (un impuls de tipul "low-pulse").
2. **Întrerupere:** DSP-ul detectează IA, oprește orice altă activitate și citește valoarea $x(n)$ din registrul de intrare.
3. **Procesare:** Se aplică algoritmul ales (ALE, AGC sau MF).
4. **Ieșire ($y(n)$):** Valoarea rezultată este scrisă în buffer-ul de ieșire.

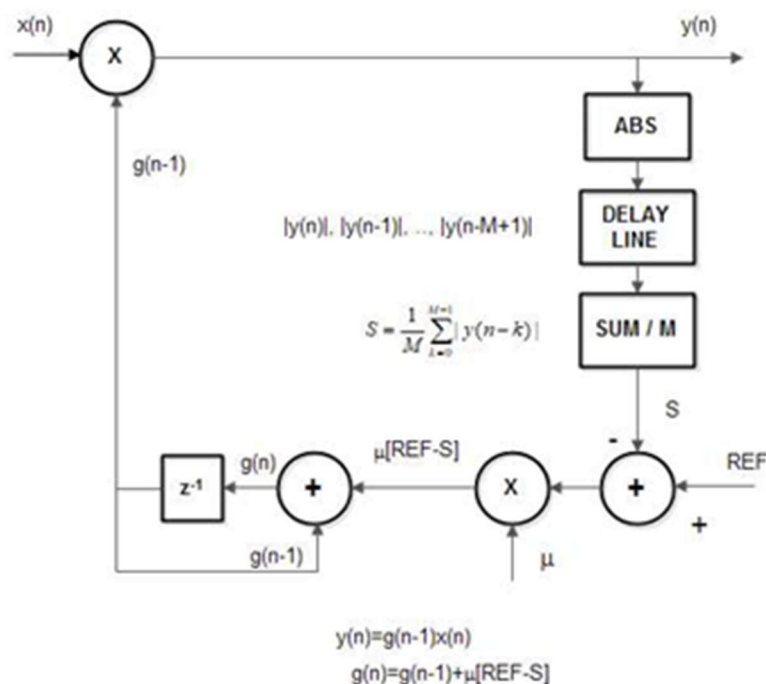
3. Descrierea funcțiilor de prelucrare digitală a semnalului:

Descrierea teoretică a funcțiilor folosite este prezentată mai jos. În etapa actuală, *toate funcțiile realizează bypass* ($y(n) = x(n)$).

3.1. Automatic Gain Control (AGC).

Funcția are rolul de a menține nivelul semnalului de la ieșire relativ constant, indiferent de variațiile nivelului semnalului de la intrare. Sistemul calculează valoarea medie a modului semnalului de la ieșire (S) pe o fereastră de M eșantioane. Această medie este comparată cu o valoare de referință (REF).

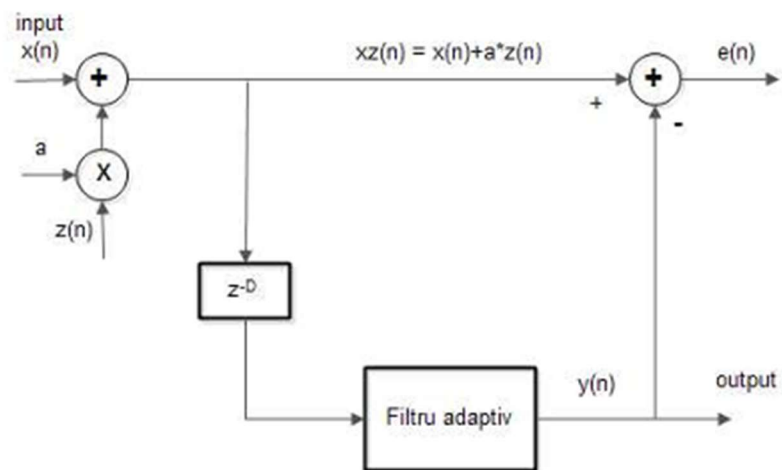
Parametri: REF, M, μ



3.2. Adaptive Line Enhancer (ALE).

Funcția este utilizată pentru a extrage sau a amplifica semnale periodice care sunt expuse unui zgomot de bandă largă. Se folosește un filtru adaptiv și o linie de întârziere (de întârziere D) aleasă astfel încât zgomotul (aleatoriu) să devină necorelat, în timp ce partea periodică a semnalului să rămână corelată.

Parametri: k, a, μ, λ

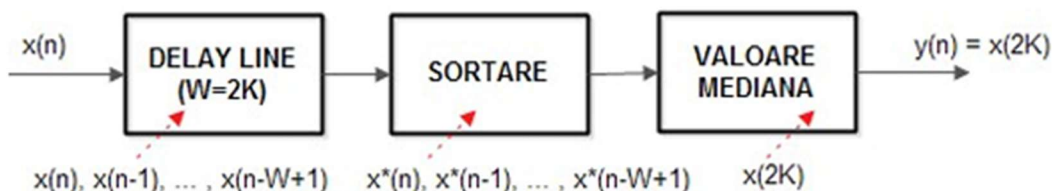


$$y(n) = \sum_{k=0}^{N-1} h(k) xz(n-k-D)$$

$$h(k) = (1 - \mu\lambda)h(k) + \mu xz(n-k-D)e(n) \quad k=0, 1, \dots, N-1$$

3.3. Median Filter (MF).

Acesta este un filtru neliniar, extrem de eficient în eliminarea zgomotului de timp impuls (vârfuri bruște de scurtă durată). Se preia o fereastră de $W = 2K$ eșantioane din semnal (unde K – dimensiunea ferestrei de filtrare), eșantioane ce vor fi sortate crescător. Valoarea de la ieșire (semnalul $y(n)$) va fi astfel valoarea mediană (cea din mijlocul listei sortate).



Parametru: K

4. Schema electrică a subsistemului ARM:

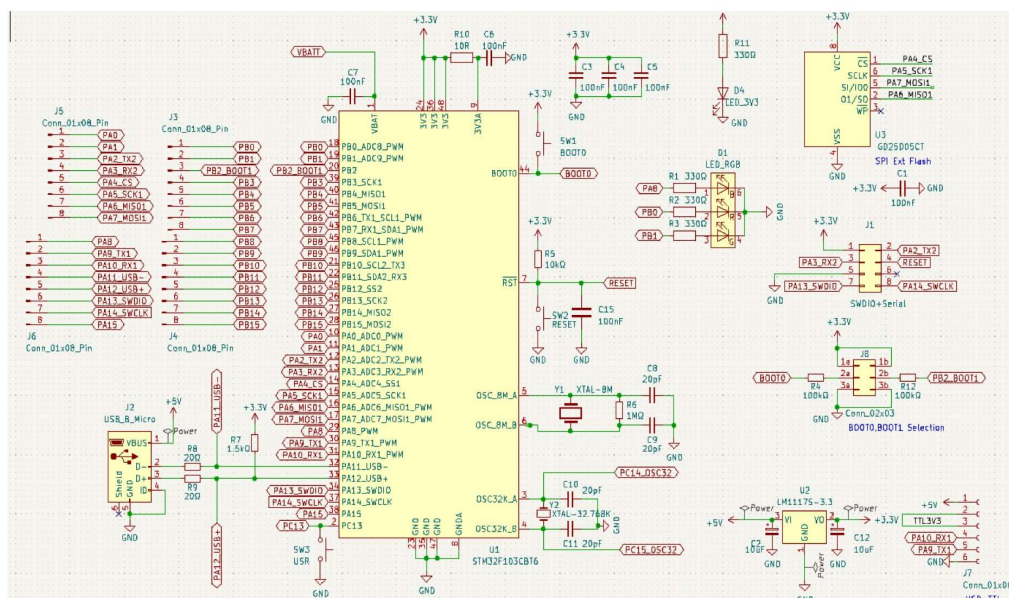


figura 3 – schema electrică STM32

Placa STM32 folosește structura microcontrolerului STM32F103CBT6.

Caracteristicile acestuia sunt:

- **J3,J4,J5,J6 – GPIO Headers** = conectori de expansiune pentru pinii GPIO ai microcontrolerului, aceștia pot fi folosiți pentru senzori, comunicare
- **J2 – USB Micro-B** = conectare la PC pentru programare/alimentare -> D+, D- trec prin R7, R8 pentru compatibilitatea USB (protecție ESD)
- **J1** = conector de 8 pini care combina interfața de debug/programare SWD (serial wire debug) și interfata UART (serială)
- **PA2_TX2** = UART Tx (iesire din STM32 către PC)
- **PA3_RX2** = UART Rx (intrare de la PC către STM32)
 - **PA13_SWDIO** = linie SWDIO date/debug (esential pentru STM32CUBEIDE - upload din PC trimite firmware prin usb la st-link, iar st-link trimite date pe SWDIO si sincronizeaza prin SWCLK, iar STM32 primește codul și îl scrie în Flash)

- **PA14_SWCLK** = linie SWCLK ceas debug

U2 = regulator de tensiune

- *intrare de +5V - iesire +3.3V* pentru alimentarea microcontrolerului
- **C12+C13** stabilizează tensiunea de iesire
- **D2 - Dioda Schottky** = protejează circuitul la conectarea inversa a alimentării
- **Y1** = oscilator principal de 8MHz
- **Y2** = oscilator RTC de 32.768kHz (condesatoarele si rezistentele asigura functionare stabila)

U3 = Memorie SPI Flash externa

- **memorie non-volatila** (nu isi pierde continutul atunci cand se intrerupe alimentarea cu energie) conectata pe SPI
- stocare de date suplimentare
- include pin WP (write protect) si HOLD

STM32 are o memorie Flash interna (non-volatila) pentru salvarea codului - firmware.

- **memoria suplimentara** = SPI Flash (Serial Peripheral Interface) = protocol de comunicatie seriala sincronă
- **MOSI** = Master Out Slave in
- **MISO** = Master In Slave Out

BOOT0 = determina sursa de boot(Flash/RAM/UART (J8 determina selectia))

- **SW2** - RESET manual (R5 rezistență de pull-up)

5. Schema electrică a subsistemului DSP:

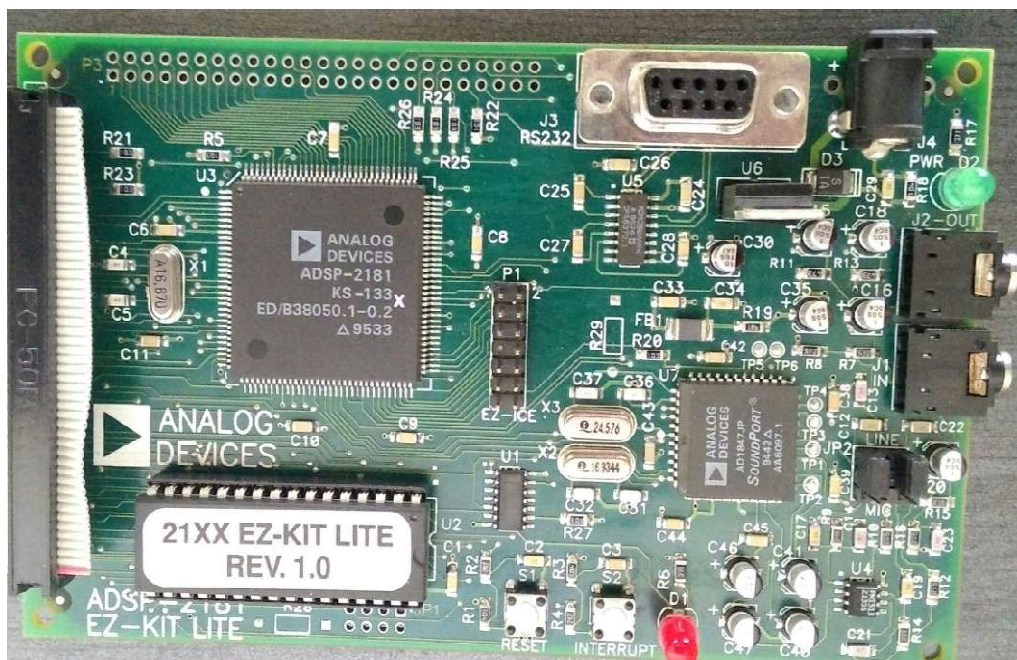


figura 4 – placa de evaluare ADSP-2181 EZ-Kit Lite (vedere fizică)

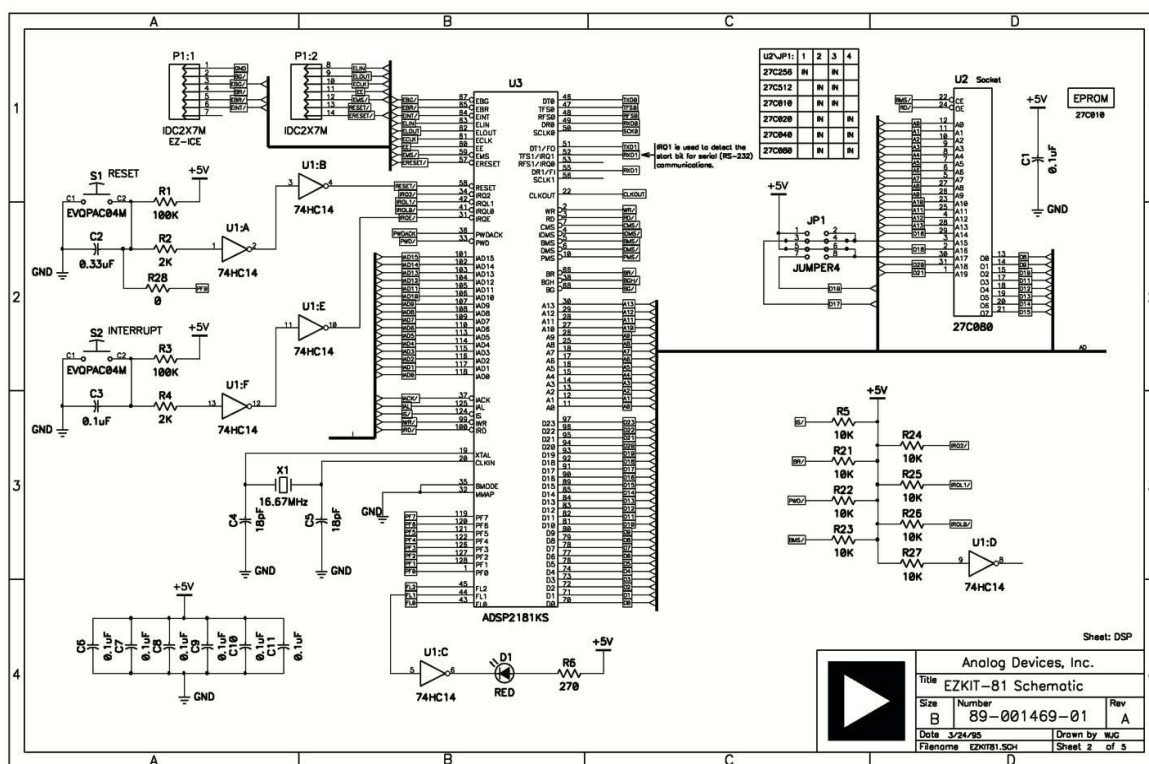


figura 5 – schema electrică EZKIT-81 – DSP

CPU (ADSP2181KS)

- gestionează toate operațiunile digitale
- nucleu optimizat pentru calcule rapide (RISC)
- magistrale de date și adrese

P1, P2 - interfața de depanare EZ-ICE

- port pentru emulatorul EZ-ICE (depanarea codului direct pe Hardware)

S1 RESET - circuit de reset

- reset hardware pentru DSP prin apăsarea lui S1
- 74HC14 - inversor Schmitt Trigger (asigura stabilitatea semnalului de reset)
- R1 menține linia de reset la +5V (HIGH)
- C2 creează o scurtă întârziere (S1 apăsat)
- C2 se încarcă prin R1 (S1 *eliberate*)

S2 INTERRUPT - circuit de întrerupere externă

- activarea întreruperii externe prin apăsarea S2
- nu consumă constant resurse
- C3 => filtru în trecerile HIGH-LOW

X1 - oscilator

- frecvența de 16.67 Mhz
- semnal de tact pentru DSP
- C4 și C5 stabilizează oscilatorul
- semnalul dat de X1 este transmis la CLKIN

C6, C7, C8, C9, C10, C11 - banc de condensatoare de decuplare

- filtrarea zgomotului și stabilizarea tensiunii de alimentare
- elimină zgomotele de înaltă frecvență
- reduce interferențele electromagnetice

A0 – A14 - linii de adresă pentru selectarea memoriei

WR, RD - linii de control pentru citire/scriere în memorie

R5, R21, R24, R22, R25, R23, R26, R27 - circuit de Pull-Up pentru pinii de control

- rezistentele mențin starea logica HIGH pentru anumite semnale (SR, PWDR, IRQ)
- U1:D foloseste aceste semnale pentru a asigura stabilitatea și a preveni starile necontrolate

U2 Socket - memorie EPROM

- nevolatila, folosită pentru a stoca programul firmware al DSP-ului
- selectata prin JP1 - permite configurarea dimensiunii memoriei
- CE - Chip Enable (activeaza memoria)
- OE - Output Enable (activeaza ieșirea de date)
- A0-A19 - magistrala de adrese (2^{20} locatii de memorie)
- O0-O7 - output data

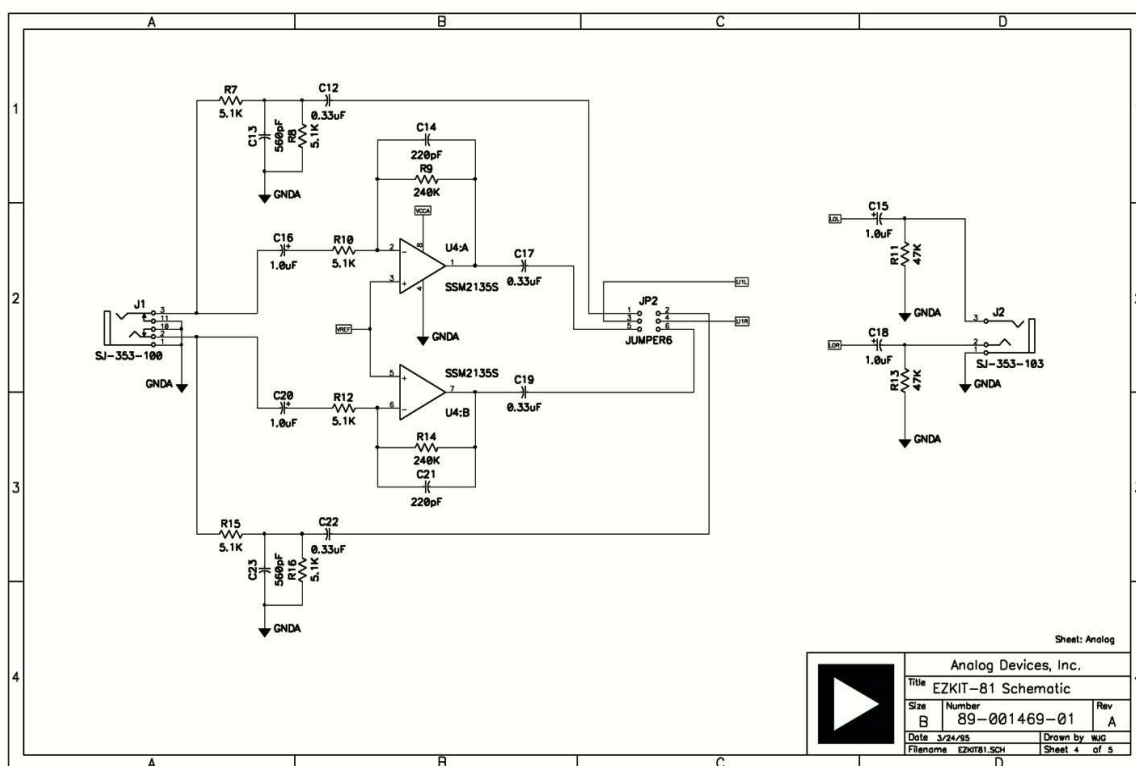


figura 6 – schema electrica EZKIT-81 – CODEC

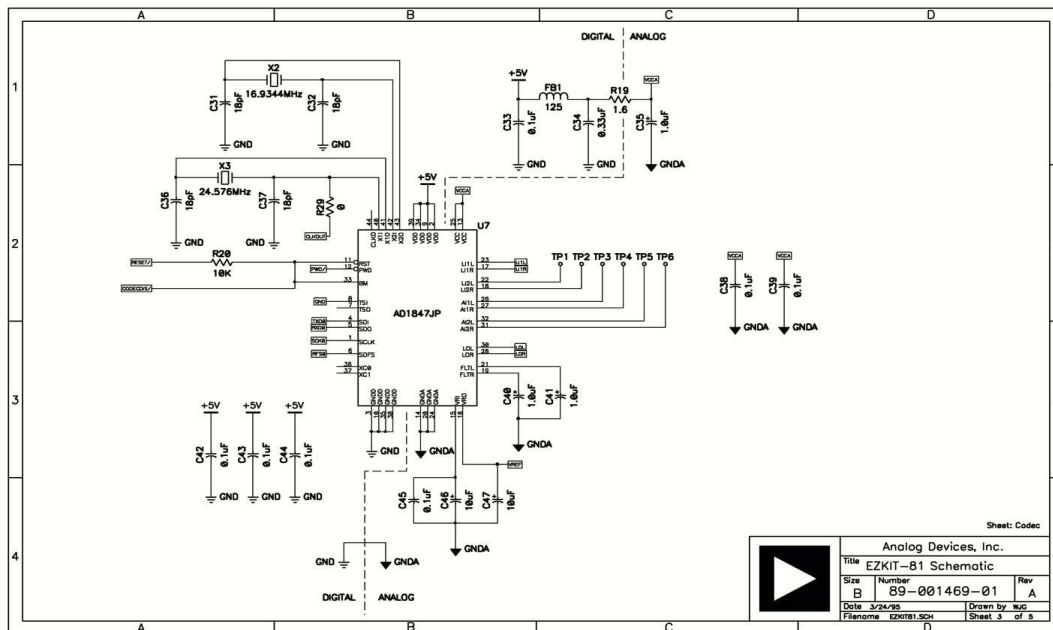


figura 7 – schema electrica EZKIT-81 – CODEC

AD1847JP = CODEC audio-stereo

- face conversia dintre un semnal digital (folosit de DSP) si un semnal analogic (căști/difuzoare)

L1R, L1L - intrări audio stânga/dreapta

L2R, L2L - a doua pereche de intrări audio

A1L, A2R - linii pentru microfoane sau alte surse audio

LOL, LOR - ieșiri audio analogice (difuzoare, casti)

- FLTR - filtru analogic

X2, X3 - Memorie EPROM

- nevolatilă, folosita pentru a stoca programul firmware al DSP-ului
- selectată prin JP1 - permite configurarea dimensiunii memoriei
- CE - Chip Enable (activează memoria)
- OE - Output Enable (activează iesirea de date)
- A0-A19 - magistrala de adrese (2^{20} locatii de memorie)
- O0-O7 - output data

RESET

- mentinut la HIGH prin R20

TXD0, RXD0, SCK0, RFS0 - linii de control pentru interfata digitala

- SDI = Serial Data Input - intrare de date digitale (de la DSP)
- SDO = Serial Data Output - ieșire de date digitale (catre DSP)
- SCLK = Serial Clock - semnal de sincronizare pentru transmitere date
- SOFS = Sync Frame – indică inceputul unui pachet de date audio

Filtru de separare intre alimentarea digitală si cea analogica

- FB1 = ferrite bead - blocheaza zgomotul de inalta frecventa inainte sa ajunga la partea analogica
- C35 actioneaza ca un rezervor de energie (tensiune stabila)

Filtru de stabilizare pt tensiunea de referinta VREF

- stabilizează tensiunea de referinta necesara in conversia analog-digital si digital-analog
- C45 filtreaza frecvențe inalte
- C46 și C47 - filtrare frecvenței medii și reglare a VREF

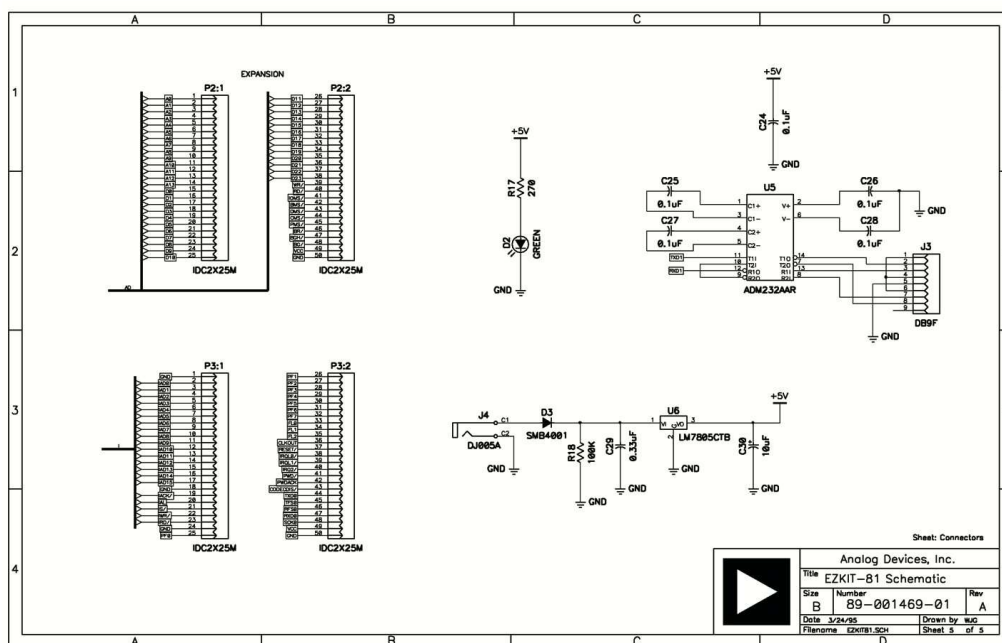
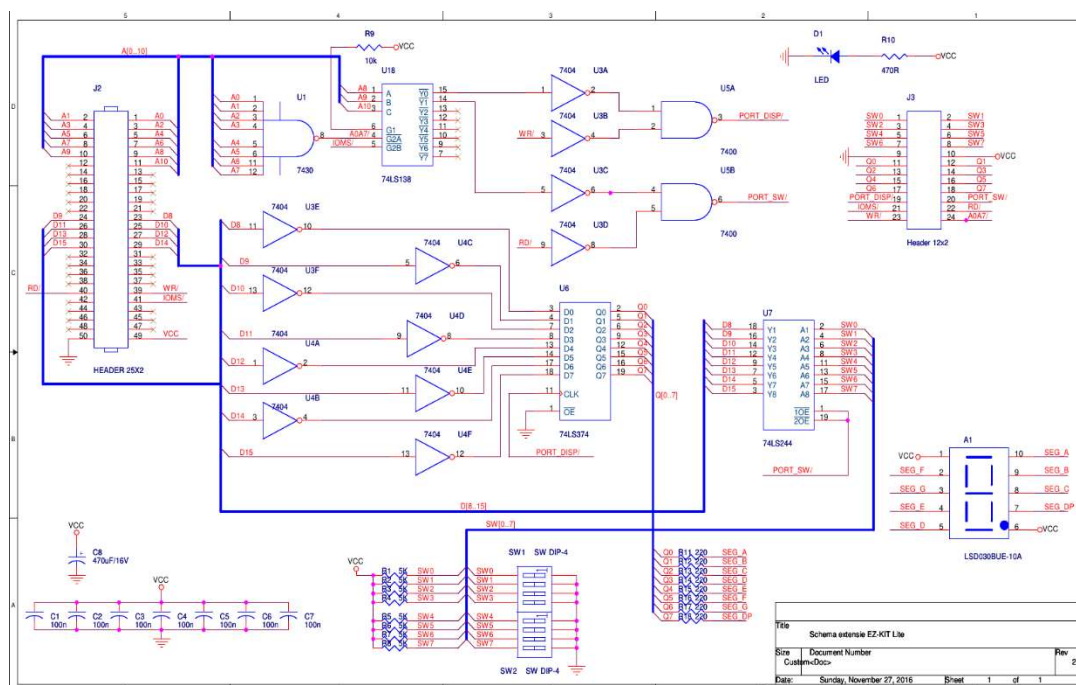
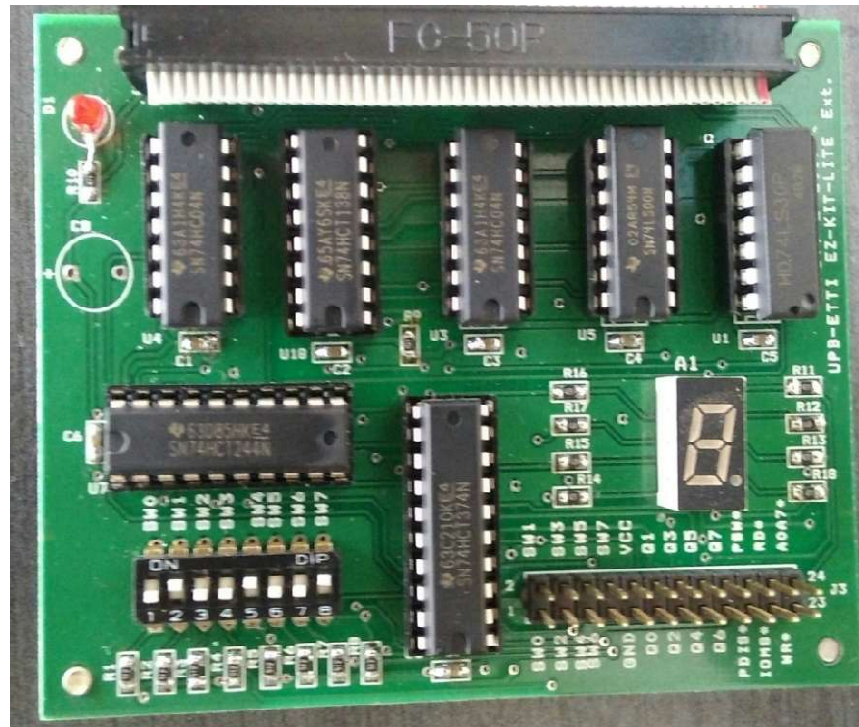


figura 8 – schema electrica EZKIT-81 – Conectori

6. Schema electrică a extensiei IO:

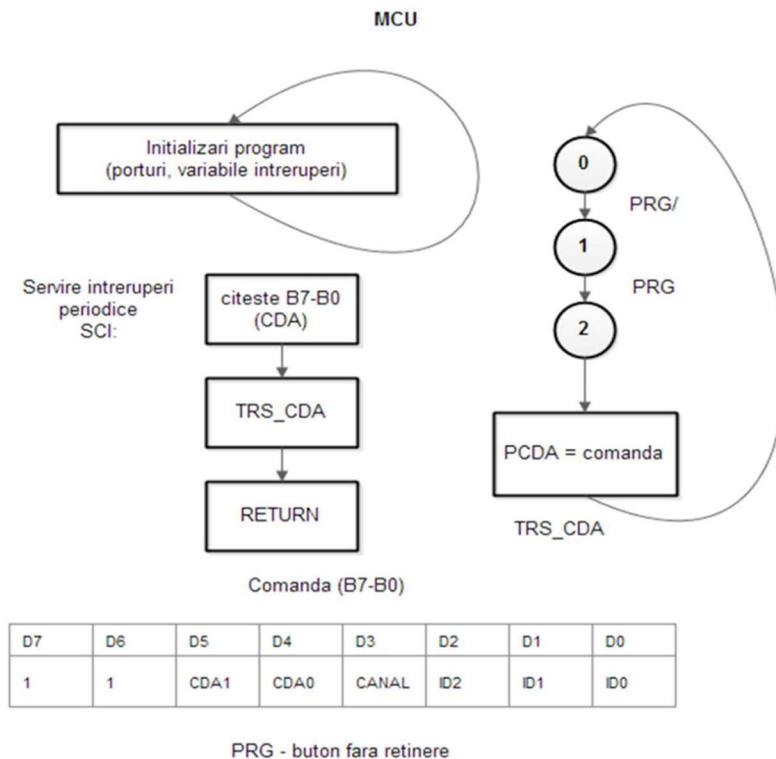


Circuite Integrate (IC-uri) și Componente:

- **7404 (U3, U4):** Porți **NOT** (inversoare). Invertesc semnalele de control și adresele pentru a asigura polaritatea corectă necesară activării celorlalte chip-uri.
- **7430 (U1):** Poartă **NAND cu 8 intrări**. Verifică liniile de adresă superioare; generează o ieșire 'low' doar când adresa selectată corespunde acestui modul de extensie.
- **74LS138 (U18): Decodor 3-la-8 linii**. Selectează sub-modulele extensiei (afișajul sau butoanele) prin activarea unei singure ieșiri în funcție de adresele A8, A9 și A10.
- **74LS374 (U6): Registru octal cu latch**. Stochează datele de pe magistrală pentru a menține aprinse segmentele afișajului LED, chiar și după ce procesorul trece la altă sarcină.
- **74LS244 (U7): Buffer octal (Tri-state)**. Acționează ca o poartă care permite trecerea stării butoanelor către procesor doar atunci când acesta execută o comandă de citire (PORT_SW/).
- **7400 (U5):** Porți **NAND cu 2 intrări**. Combină semnalele de selecție cu semnalele de scriere/citire (WR/, RD/) pentru a genera semnalele finale de activare a porturilor.
- **Afișaj LED (A1 - LSD030BUE):** Afișor cu 7 segmente plus punct decimal. Primește datele stocate de U6 prin rezistențe de limitare pentru a afișa cifre sau caractere.
- **Comutatoare (SW1, SW2):** Două blocuri de câte 4 switch-uri (DIP-4) care permit setarea manuală a 8 biți de date (SW0-SW7) pentru a interacționa cu programul DSP-ului.

7. Subsistemul ARM – descriere conceptuală/funcțională

7.1. Descrierea mașinii de stări TRS_CDA



STARE 0 – ELIBERARE PRG: Se verifică dacă butonul PRG (PA9) este eliberat. Dacă da, se trece în starea 1. NU se va accepta o comandă nouă cât butonul este ținut apăsat.

STARE 1 – APĂSARE PRG: La eliberarea butonului PRG, se vor citi butoanele B0-B7 (shift dreapta cu 1, mască 0x3F pentru 6 biți). Se construiește comanda: $comanda_dsp = 0xC0 | date_butoane$. Se trece în starea 2.

STARE 2 – TRANSMITERE COMANDĂ: Se va transmite comanda pe magistrala PCDA către DSP (PB8-PB15). Se va afișa aceeași comandă pe LED-uri (PB0-PB7). Se revine în starea 0.

7.2. Configurare Hardware (STM32CubeMX)

- **MCU:** STM32F103CBTx (LQFP48, Cortex-M3)
- **Clock:** HSE 8 MHz → PLL ×9 → SYSCLK = 72 MHz, HCLK = 72 MHz, APB1 = 36 MHz, APB2 = 72 MHz
- **Timer TIM2:** Clock intern 72 MHz, PSC = 7199, ARR = 100, Counter Up → frecvența de întrerupere ≈ 99 Hz (~10 ms)
- **Debug:** Serial Wire (SWD)
- **GPIO intrări (fără pull-up/pull-down):** PA1–PA6 = butoane B0–B5, PA7 = B6, PA8 = B7, PA9 = PRG
- **GPIO ieșiri (push-pull, low speed):** PB0–PB7 = LED0–LED7, PB8–PB15 = PCDA0–PCDA7
- **NVIC:** TIM2 global interrupt activat, prioritate 0

CODUL C PENTRU STM32:

```
/* USER CODE BEGIN Header */
```

```
/**
```

```
*****
```

```
*****
```

```
* @file      : main.c
```

```
* @brief     : Main program body
```

```
*****
```

```
*****
```

```
* @attention
```

```
*
```

```
* Copyright (c) 2026 STMicroelectronics.
```

```
* All rights reserved.
```

*

* This software is licensed under terms that
can be found in the LICENSE file

* in the root directory of this software
component.

* If no LICENSE file comes with this
software, it is provided AS-IS.

*

*/

/* USER CODE END Header */

/* Includes

-----*/

#include "main.h"

/* Private includes

-----*/

/* USER CODE BEGIN Includes

*/

/* USER CODE END Includes */

/* Private typedef

-----*/

```

/* USER CODE BEGIN PTD */

/* USER CODE END PTD */


/* Private define
-----*/

/* USER CODE BEGIN PD */
/* USER CODE END PD */


/* Private macro
-----*/

/* USER CODE BEGIN PM */
/* USER CODE END PM */


/* Private variables
-----*/

TIM_HandleTypeDef htim2;


/* USER CODE BEGIN PV */
uint8_t comanda_dsp = 0;    // comanda de 8 biti transmisa catre DSP
uint8_t stare_trs = 0;    // starea masinii de stari TRS_CDA (0, 1, 2)
/* USER CODE END PV */


/* Private function prototypes -----*/
void SystemClock_Config(void);

```

```
static void MX_GPIO_Init(void);
```

```
static void MX_TIM2_Init(void);
```

```
/* USER CODE BEGIN PFP */
```

```
/* USER CODE END PFP */
```

```
/* Private user code
```

```
-----*/
```

```
/* USER CODE BEGIN 0 */
```

```
/*
```

```
=====
```

EXPLICATIE FSM (FINITE STATE MACHINE - Masina de stari)

Aceasta functie este un ISR (Interrupt Service Routine). Se executa periodic, la fiecare expirare a Timer-ului 2 (setat mai jos la 100 de perioade).

Ea implementeaza masina de stari TRS_CDA care citeste butoanele si trimite

datele catre un DSP si catre LED-uri.

```
=====
```

```
===== */
```

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
```

```
{
```

```
if (htim->Instance == TIM2)
```

```
{
```

```

// Citeste starea curenta a butonului PRG (aflat pe pinul PA9)
// Acest buton controleaza tranzitiile masinii de stari.
GPIO_PinState prg = HAL_GPIO_ReadPin(GPIOA, PRG_Pin);

switch (stare_trs)
{
    case 0:
        // STAREA 0: Starea de repaus/asteptare.
        // Scopul: ne asiguram ca butonul PRG a fost ELIBERAT complet
        // inainte de a inregistra o noua comanda, prevenind citirile multiple.
        if (prg == GPIO_PIN_RESET) // Daca PRG e eliberat (logic 0)
        {
            stare_trs = 1; // Trecem in starea 1 pentru a astepta o noua apasare
        }
        break;

    case 1:
        // STAREA 1: Asteptam apasarea butonului PRG.
        // Cand utilizatorul apasa PRG, citim starea butoanelor de date.
        if (prg == GPIO_PIN_SET) // Daca PRG este apasat (logic 1)
        {
            // Citim tot portul A. Butoanele B0-B5 sunt conectate la pinii PA1-PA6.
            uint16_t stare_port_A = GPIOA->IDR;

```

```

// Shiftam la dreapta cu 1 bit (astfel PA1 ajunge pe pozitia bitului 0)
// si aplicam masca 0x3F (0011 1111 in binar) pentru a izola fix cei 6 biti
utili (B0-B5).
uint8_t date_butoane = (uint8_t)((stare_port_A >> 1) & 0x3F);

// Construim comanda de 8 biti pentru DSP.
// Format cerut: D7=1, D6=1, D5-D0=datele de pe butoane.
// 0xC0 inseamna 1100 0000 in binar. Prin operatia SAU logic (|), setam
primii 2 biti pe 1.
comanda_dsp = 0xC0 | date_butoane;

stare_trs = 2; // Datele sunt gata, trecem in starea 2 pentru a le trimite.
}
break;

case 2:
// STAREA 2: Transmiterea comenzii catre hardware.
// Folosim portul B pentru iesiri (setat complet in MX_GPIO_Init).
// - Octetul superior al Portului B (PB8-PB15) = pinii PCDA7-PCDA0 -> se
duc catre DSP.
// - Octetul inferior al Portului B (PB0-PB7) = pinii LED7-LED0 -> afiseaza
vizual comanda.

// Explicatie operatie pe biti:
// (comanda_dsp << 8) pune cei 8 biti de date in jumatatea de sus (pentru
DSP).

```

```

// | (comanda_dsp) pune aceiasi 8 biti in jumatatea de jos (pentru LED-uri).
GPIOB->ODR = ((uint16_t)comanda_dsp << 8) | (uint16_t)comanda_dsp;

    stare_trs = 0; // Am terminat actiunea. Ne intoarcem in starea 0 pentru a
astepta eliberarea PRG.

    break;

default:

    stare_trs = 0; // Mecanism de siguranta: daca ajungem intr-o stare
necunoscuta, resetam la 0.

    break;

}

}

}

/* USER CODE END 0 */

/**
 * @brief The application entry point.
 * @retval int
 */
int main(void)
{
    /* USER CODE BEGIN 1 */
    /* USER CODE END 1 */

```



```

/* MCU Configuration-----*/

// 1. INITIALIZARE HAL: Reseteaza toate perifericele, initializeaza interfata
Flash si Systick-ul.

HAL_Init();

/* USER CODE BEGIN Init */

/* USER CODE END Init */

// 2. INITIALIZARE CEAS: Configureaza oscilatorul (HSE/PLL) pentru a face
microcontrolerul sa ruleze la 72MHz.

SystemClock_Config();

/* USER CODE BEGIN SysInit */

/* USER CODE END SysInit */

// 3. INITIALIZARE PINI (GPIO): Configureaza care pini sunt intrari si care sunt
iesiri (detalii in functia de jos).

MX_GPIO_Init();

// 4. INITIALIZARE TIMER: Seteaza parametrii (prescaler, perioada) pentru
Timer 2, care va declansa FSM-ul.

MX_TIM2_Init();

/* USER CODE BEGIN 2 */

// 5. PORNIRE TIMER IN REGIM DE INTRERUPERI:

```

```

// Aici se activeaza efectiv Timer 2. De fiecare data cand numara pana la
// limita,
// va apela automat functia HAL_TIM_PeriodElapsedCallback de mai sus.
HAL_TIM_Base_Start_IT(&htim2);
/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */

    // Bucla infinita ramane goala deoarece toata logica programului (FSM-ul)
    // este tratata in fundal prin intreruperile generate de Timer 2.

    /* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
}

/**
 * @brief System Clock Configuration
 * @retval None
 */
void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};

```

```

RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

// Seteaza utilizarea cristalului extern (HSE) si activarea PLL-ului
RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
RCC_OscInitStruct.HSEState = RCC_HSE_ON;
RCC_OscInitStruct.HSEPredivValue = RCC_HSE_PREDIV_DIV1;
RCC_OscInitStruct.HSIState = RCC_HSI_ON;
RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
RCC_OscInitStruct.PLL.PLLMUL = RCC_PLL_MUL9; // Frecventa de baza (ex:
8MHz) * 9 = 72MHz frecventa sistem

if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK) { Error_Handler(); }

// Configureaza divizoarele de ceas pentru diferitele magistrale (AHB, APB1,
APB2)

RCC_ClkInitStruct.ClockType =
RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
        |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) !=
HAL_OK) { Error_Handler(); }
}

```

```

/**
 * @brief TIM2 Initialization Function
 */
static void MX_TIM2_Init(void)
{
    TIM_ClockConfigTypeDef sClockSourceConfig = {0};
    TIM_MasterConfigTypeDef sMasterConfig = {0};
    TIM_OC_InitTypeDef sConfigOC = {0};

    htim2.Instance = TIM2;

    // Prescaler si Period determina cat de des se va declansa intreruperea FSM-ului
    htim2.Init.Prescaler = 7199;
    htim2.Init.Period = 100;
    htim2.Init.CounterMode = TIM_COUNTERMODE_UP;
    htim2.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
    htim2.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
    if (HAL_TIM_Base_Init(&htim2) != HAL_OK) { Error_Handler(); }

    sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
    if (HAL_TIM_ConfigClockSource(&htim2, &sClockSourceConfig) != HAL_OK) {
        Error_Handler(); }
    if (HAL_TIM_OC_Init(&htim2) != HAL_OK) { Error_Handler(); }
}

```

```

sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
if (HAL_TIMEx_MasterConfigSynchronization(&htim2, &sMasterConfig) !=
HAL_OK) { Error_Handler(); }

sConfigOC.OCMode = TIM_OCMODE_TIMING;
sConfigOC.Pulse = 0;
sConfigOC.OCpolarity = TIM_OCPOLARITY_HIGH;
sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;
if (HAL_TIM_OC_ConfigChannel(&htim2, &sConfigOC, TIM_CHANNEL_1) !=
HAL_OK) { Error_Handler(); }

HAL_TIM_MspPostInit(&htim2);
}

/**
 * @brief GPIO Initialization Function
 * @details Aici se face initializarea fizica a pinilor (Intrari / Iesiri)
 */
static void MX_GPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};

    /* 1. ACTIVARE CEAS PENTRU PORTURI */

    // Pentru ca pinii sa functioneze, trebuie alimentate porturile respective cu
    semnal de ceas.

    __HAL_RCC_GPIOD_CLK_ENABLE();

```

```
__HAL_RCC_GPIOA_CLK_ENABLE();
```

```
__HAL_RCC_GPIOB_CLK_ENABLE();
```

```
/* 2. INITIALIZARE STARE IESIRI (PORT B) */
```

```
// Setam toti pinii de pe Portul B pe 0 logic (LOW) la pornire, pentru a tine  
LED-urile stinse initial.
```

```
HAL_GPIO_WritePin(GPIOB, LED0_Pin|LED1_Pin|LED2_Pin|PCDA2_Pin  
|PCDA3_Pin|PCDA4_Pin|PCDA5_Pin|PCDA6_Pin  
|PCDA7_Pin|LED3_Pin|LED4_Pin|LED5_Pin  
|LED6_Pin|LED7_Pin|PCDA0_Pin|PCDA1_Pin, GPIO_PIN_RESET);
```

```
/* 3. CONFIGURARE INTRARI (PORT A - BUTOANE) */
```

```
// Pinii B0-B7 si PRG sunt configurati ca INTRARI pentru a citi starea  
butoanelor.
```

```
GPIO_InitStruct.Pin = B0_Pin|B1_Pin|B2_Pin|B3_Pin  
|B4_Pin|B5_Pin|B6_Pin|B7_Pin  
|PRG_Pin;
```

```
GPIO_InitStruct.Mode = GPIO_MODE_INPUT;    // Setat ca intrare digitala
```

```
GPIO_InitStruct.Pull = GPIO_NOPULL;        // Fara rezistente interne (potrivit  
pentru simulator)
```

```
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);    // Aplica setarile pe Portul A
```

```
/* 4. CONFIGURARE IESIRI (PORT B - LED-uri si Semnale DSP) */
```

```
// Toti pinii de pe Portul B sunt configurati ca IESIRI.
```

```
GPIO_InitStruct.Pin = LED0_Pin|LED1_Pin|LED2_Pin|PCDA2_Pin
```

```

|PCDA3_Pin|PCDA4_Pin|PCDA5_Pin|PCDA6_Pin
|PCDA7_Pin|LED3_Pin|LED4_Pin|LED5_Pin
|LED6_Pin|LED7_Pin|PCDA0_Pin|PCDA1_Pin;

GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP; // Setat ca iesire de tip
Push-Pull (poate furniza curent pentru LED)

GPIO_InitStruct.Pull = GPIO_NOPULL;

GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW; // Viteza mica de
comutare e suficienta

HAL_GPIO_Init(GPIOB, &GPIO_InitStruct); // Aplica setarile pe Portul B
}

/**
 * @brief This function is executed in case of error occurrence.
 */
void Error_Handler(void)
{
    __disable_irq();
    while (1)
    {
    }
}

#ifdef USE_FULL_ASSERT
void assert_failed(uint8_t *file, uint32_t line)
{
}

#endif /* USE_FULL_ASSERT */

```

7.3. Simulări în programul STM32CubeIDE

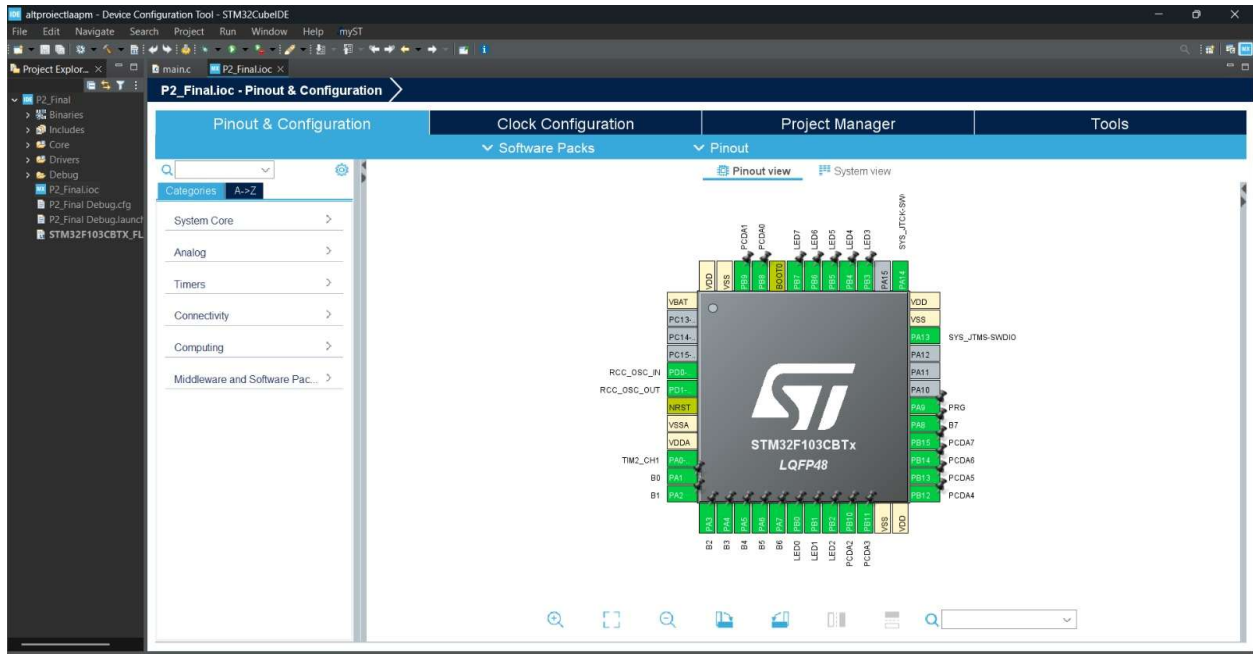


figura 11 – configurarea pinilor (pinout view) – STM32CUBEMX

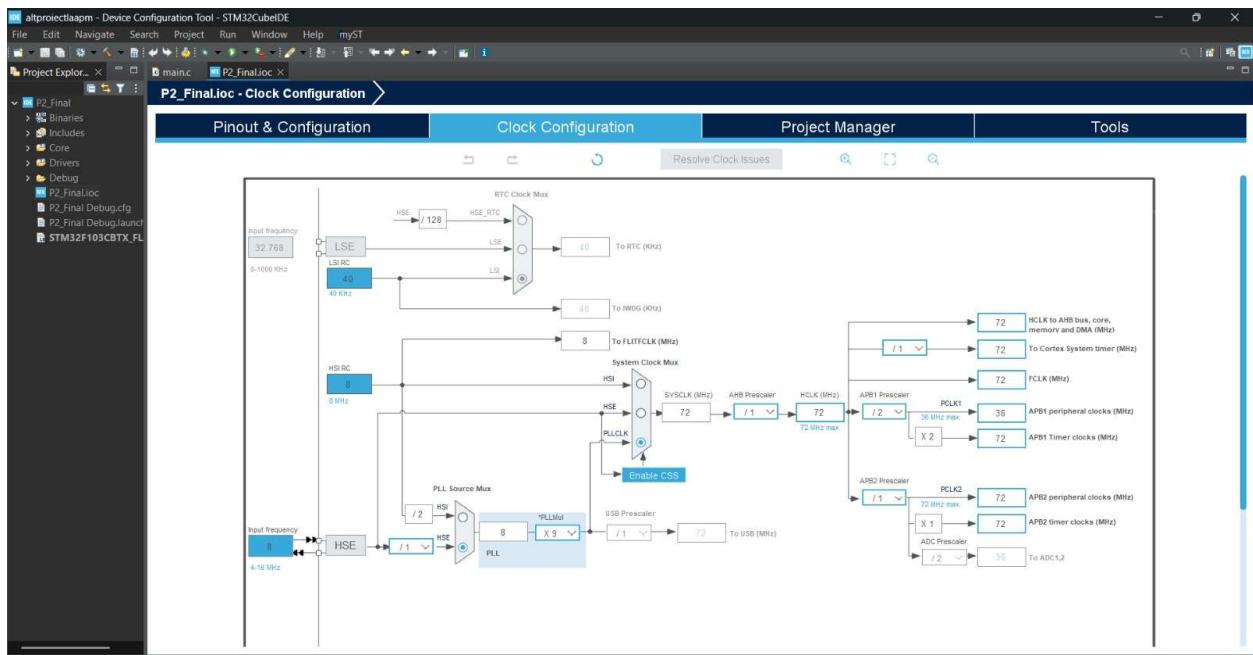


figura 12 – configurarea arborelui de ceas (clock configuration)

Pin ...	Signal o...	GPIO o...	GPIO m...	GPIO P...	Maximu...	User La...	Modified
PA1	n/a	n/a	Input mo...	No pull-...	n/a	B0	✓
PA2	n/a	n/a	Input mo...	No pull-...	n/a	B1	✓
PA3	n/a	n/a	Input mo...	No pull-...	n/a	B2	✓
PA4	n/a	n/a	Input mo...	No pull-...	n/a	B3	✓
PA5	n/a	n/a	Input mo...	No pull-...	n/a	B4	✓
PA6	n/a	n/a	Input mo...	No pull-...	n/a	B5	✓
PA7	n/a	n/a	Input mo...	No pull-...	n/a	B6	✓
PA8	n/a	n/a	Input mo...	No pull-...	n/a	B7	✓
PA9	n/a	n/a	Input mo...	No pull-...	n/a	PRG	✓

figura 13 – pinout PA1-PA8 (B0-B7) și PA9 (PRG)

PB0	n/a	Low	Output ...	No pull-...	Low	LED0	✓
PB1	n/a	Low	Output ...	No pull-...	Low	LED1	✓
PB2	n/a	Low	Output ...	No pull-...	Low	LED2	✓
PB3	n/a	Low	Output ...	No pull-...	Low	LED3	✓
PB4	n/a	Low	Output ...	No pull-...	Low	LED4	✓
PB5	n/a	Low	Output ...	No pull-...	Low	LED5	✓
PB6	n/a	Low	Output ...	No pull-...	Low	LED6	✓
PB7	n/a	Low	Output ...	No pull-...	Low	LED7	✓
PB8	n/a	Low	Output ...	No pull-...	Low	PCDA0	✓

figura 14 – pinout PB0-PB7 (LED0-7) și PB8 (PCDA0)

PB9	n/a	Low	Output ...	No pull-...	Low	PCDA1	✓
PB10	n/a	Low	Output ...	No pull-...	Low	PCDA2	✓
PB11	n/a	Low	Output ...	No pull-...	Low	PCDA3	✓
PB12	n/a	Low	Output ...	No pull-...	Low	PCDA4	✓
PB13	n/a	Low	Output ...	No pull-...	Low	PCDA5	✓
PB14	n/a	Low	Output ...	No pull-...	Low	PCDA6	✓
PB15	n/a	Low	Output ...	No pull-...	Low	PCDA7	✓

figura 15 – pinout PB9-PB15 (PCDA1-7)

8. Subsistemul DSP – descriere conceptuală/funcțională

8.1. Structura programului:

Programul este scris în limbaj de asamblare ADSP-21xx și este organizat în secțiuni de memorie conform arhitecturii Harvard:

- **buf_var1 (DM):** buffer de recepție rx_buf[3] — stare + canal stâng + canal drept
- **buf_var2 (DM):** buffer de transmisie tx_buf[3] — comandă + canal stâng + canal drept
- **buf_var3 (DM):** cele 13 cuvinte de inițializare a codec-ului AD1847
- **data1 (DM):** variabilele programului (Flag_CDA, cmd_word, cmd_valida, id_functie, param_id, canal, x_in, y_out, PF_input, PF_output, stat_flag)
- **interrupts (PM):** tabela vectorilor de întrerupere
- **seg_code (PM):** codul principal

8.2. Inițializarea sistemului:

SPORT0: Se configurează autobuffering-ul, modul multicanal pe 16 biți, cu codec-ul furnizând ceasul. Bufferele circulare sunt configurate: i5/l5 pentru rx_buf, i6/l6 pentru tx_buf, i3/l3 pentru init_cmds.

Codec AD1847: Se trimite secvențial 13 cuvinte de comandă prin SPORT0 TX. Se așteaptă finalizarea auto-calibrării (bitul ACI trece 0→1→0 în registrul de stare). Se dezactivează mute-ul pe canalele de ieșire.

Porturi PF: PF0–PF3 configurate ca intrări, PF4–PF7 ca ieșiri (Prog_Flag_Comp_Sel_Ctrl = 0x00F0).

Întreruperi: imask = b#0010100001 activează Timer (bitul 9), SPORT0 RX (bitul 5) și IRQLO (bitul 0).

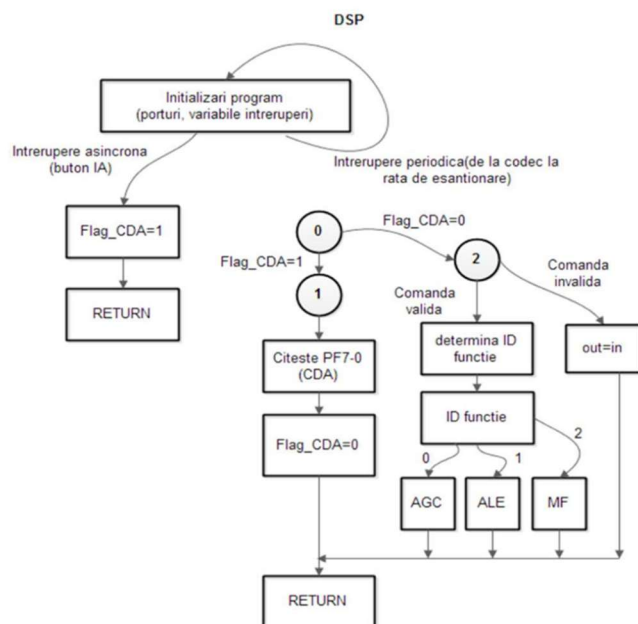
8.3. Întreruperile active:

IRQLO (isr_ia): Întreruperea asincronă de la butonul IA. Setează Flag_CDA = 1 pentru a semnaliza mașinii de stări că o comandă nouă este disponibilă.

Timer (isr_periodic): Rulează mașina de stări DSP la fiecare declanșare.

SPORT0 RX (input_samples): La fiecare eșantion audio recepționat de la codec: citește rx_buf+2 în x_in, scrie y_out în tx_buf+1 și tx_buf+2 (ambele canale), realizează testul I/O și actualizează porturile PF.

8.4. Mașina de stări DSP:



STARE 0 – RECEPȚIONARE/DECIZIE: Verifică dacă Flag_CDA != 0 (primește comandă nouă), sare la starea 1. Dacă Flag_CDA == 0, sare la starea 2.

STARE 1 – CITIREA COMENZII: Citește Prog_Flag_Data, folosind masca 0x00FF și salvează rezultatul în cmd_word. Resetează Flag_CDA = 0. Execută returnare (rti).

STARE 2 – VALIDARE ȘI EXTRAGERE A PARAMETRILOR + EXECUȚIE:

- validare: validează mesajul oferit (cmd_word & 0x00C0) == 0x00C0?, dacă nu, atunci efectuează returnare (rti).
- extragere CDA: primește id_functie (0 = AGC, 1 = ALE, 2 = MF)
- extragere parametru: param_id (0-7) din cmd_word & 0x0007
- extragere canal: canal (0 sau 1) din cmd_word & 0x0008
- rutare(id_functie): 0 - do_agc, 1 - do_ale, 2 - do_mf

CODUL ÎN LIMBAJ DE ASAMBLARE A DSP:

```
#include "def2181.h"
```

```
#define PORT_OUT 0xFF
```

```
#define PORT_IN 0x1FF
```

```
.SECTION/DM buf_var1;
```

```
.var/circ rx_buf[3];
```

```
.SECTION/DM buf_var2;
```

```
.var/circ tx_buf[3] = 0xc000, 0x0000, 0x0000;
```

```
.SECTION/DM buf_var3;
```

```
.var init_cmds[13] = 0xc002, 0xc102, 0xc288, 0xc388, 0xc488, 0xc588, 0xc680,  
0xc780, 0xc85c, 0xc909, 0xca00, 0xcc40, 0xcd00;
```

```
.SECTION/DM data1;
```

```
.var stat_flag;
```

```
.var PF_input;
```

```
.var PF_output;
```

```
.var Flag_CDA;
```

```
.var cmd_word;
```

```
.var cmd_valida;
```

```
.var id_functie;
```

```
.var x_in;
```

```
.var y_out;
```

```
.var param_id;
```

```
.var canal;
```

```
.SECTION/PM pm_da;
```

```
.SECTION/PM interrupts;
```

```
jump start;    rti; rti; rti; /*00 reset */
```

```
rti; rti; rti; rti;      /*04 IRQ2 */
```

```
rti;    rti; rti; rti;    /*08 IRQ1 */
```

```

jump isr_ia;    rti; rti; rti; /*0C IRQLO */

ar = dm(stat_flag);    /*10 SPORT0 tx */
ar = pass ar;
ifeq rti;
jump next_cmd;

jump input_samples;    /*14 SPORT0 rx*/

rti;
rti; rti;
rti;
rti; rti;
rti;    /*18 IRQE */

rti;
rti; rti; rti;    /*1C BDMA */

rti;
rti; rti; rti;    /*20 SPORT1 tx / IRQ1 */

rti;
rti; rti; rti;    /*24 SPORT1 rx / IRQ0 */

rti;
rti; rti; rti;

rti;
rti; rti; rti;

.SECTION/PM seg_code;
start:
ax0 = b#00001000000000000;
dm(Sys_Ctrl_Reg) = ax0;

```

```

i5 = rx_buf;
l5 = LENGTH(rx_buf);
i6 = tx_buf;
l6 = LENGTH(tx_buf);
i3 = init_cmds;
l3= LENGTH(init_cmds);

m1 = 1;
m5 = 1;

ax0 = b#0000110011010111;
dm(Sport0_Autobuf_Ctrl)= ax0;

ax0 = 0;
dm(Sport0_Rfsdiv) = ax0;

ax0 = 0;
dm(Sport0_Sclkdiv) = ax0;

ax0= b#1000011000001111;
dm(Sport0_Ctrl_Reg) = ax0;

ax0= b#0000000000000111;
dm(Sport0_Tx_Words0) = ax0;

ax0= b#0000000000000111;
dm(Sport0_Tx_Words1) = ax0;

ax0= b#0000000000000111;
dm(Sport0_Rx_Words0) = ax0;

ax0= b#0000000000000111;
dm(Sport0_Rx_Words1) = ax0;

ax0 = b#0001100000000000;

```

```
dm(Sys_Ctrl_Reg) = ax0;
```

```
ifc = b#00000011111110;  
nop;
```

```
icntl = b#00010;  
mstat = b#1100000;
```

```
ax0 = 1;  
dm(stat_flag) = ax0;
```

```
ena ints;  
imask = b#0001000001;
```

```
ax0 = dm(i6,m5);  
tx0 = ax0;
```

```
check_init:  
ax0 = dm(stat_flag);  
af = pass ax0;  
ifne jump check_init;  
ay0 = 2;
```

```
check_aci1:  
ax0 = dm(rx_buf);  
ar= ax0 and ay0;  
ifeq jump check_aci1;
```

```
check_aci2:  
ax0 = dm(rx_buf);  
ar= ax0 and ay0;  
ifne jump check_aci2;  
idle;
```

```
ay0 = 0xbf3f;
```

```
ax0 = dm(init_cmds + 6);  
ar = ax0 and ay0;  
dm(tx_buf) = ar;  
idle;
```

```
ax0 = dm(init_cmds + 7);  
ar = ax0 and ay0;  
dm(tx_buf) = ar;  
idle;
```

```
ifc = b#00000011111110;  
nop;
```

```
imask = b#0000100000;
```

```
si = 0xFFFF;  
dm(Dm_Wait_Reg) = si;
```

```
si = 0x00f0;  
dm(Prog_Flag_Comp_Sel_Ctrl) = si;
```

```
ax0 = 0;  
dm(PF_input) = ax0;  
dm(PF_output) = ax0;  
dm(Flag_CDA) = ax0;  
dm(cmd_word) = ax0;  
dm(cmd_valida) = ax0;  
dm(id_functie) = ax0;  
dm(x_in) = ax0;  
dm(y_out) = ax0;  
dm(param_id) = ax0;  
dm(canal) = ax0;
```

```
ena ints;
```



```
imask = b#0000100001;
```

```
imask = b#0010100000;
```

```
main_loop:
```

```
idle;
```

```
jump main_loop;
```

```
isr_ia:
```

```
ax0 = 1;
```

```
dm(Flag_CDA) = ax0;
```

```
rti;
```

```
input_samples:
```

```
ena sec_reg;
```

```
sr1 = dm(rx_buf + 2);
```

```
dm(x_in) = sr1;
```

```
si= IO(PORT_IN);
```

```
IO(PORT_OUT) = si;
```

```
ax0 = dm(Prog_Flag_Data);
```

```
ay0 = 0x000F;
```

```
ar = ax0 and ay0;
```

```
dm(PF_input) = ar;
```

```
//
```

```
=====
```

```
=====
```

```
// MASINA DE STARI
```

```
//
```

```
=====
```

```
=====
```

```
starea_0:
```

```
// Verifică dacă există o nouă comandă disponibilă (Flag_CDA)
```

```

ax0 = dm(Flag_CDA);
af= pass ax0;
ifne jump starea_1; // Dacă Flag_CDA != 0 (avem o comandă nouă), mergi în
starea_1 pentru a o citi
jump starea_2; // Dacă Flag_CDA == 0 (nu avem comandă nouă), mergi în
starea_2 pentru procesare continuă

starea_1:
// Citește cuvântul de comandă de pe portul de date și aplică o mască pentru
primii 8 biți
ax0 = dm(Prog_Flag_Data);
ay0 = 0x00FF;
ar = ax0 and ay0;
dm(cmd_word) = ar; // Salvează comanda în memorie

ax0 = 0;
dm(Flag_CDA) = ax0; // Resetează flag-ul pentru comandă nouă deoarece
tocmai a fost citită
jump output_and_return; // Scurtcircuitează procesarea pentru acest ciclu și
iese

starea_2:
// Verifică validitatea comenzii: Extrage bitii 7 și 6 (masca 0xC0)
ax0 = dm(cmd_word);
ay0 = 0x00C0;
ar= ax0 and ay0;
af = ar - ay0;
ifne jump cmd_invalida; // Dacă bitii superiori nu sunt b#11 (0xC0), comanda
nu e validă

ax0 = 1;
dm(cmd_valida) = ax0; // Setează flag-ul care indică o comandă validă

// Extrage 'id_functie' din comanda (biti 5 si 4, masca 0x30)
ax0 = dm(cmd_word);

```

```

ay0= 0x0030;
ar = ax0 and ay0;
sr= lshift ar by -4 (lo); // Shiftează dreapta cu 4 poziții
dm(id_funcție) = sr0;    // Salvează ID-ul funcției

// Extrage 'canal' din comanda (bitul 3, masca 0x08)
ax0 = dm(cmd_word);
ay0= 0x0008;
ar = ax0 and ay0;
sr= lshift ar by -3 (lo); // Shiftează dreapta cu 3 poziții
dm(canal) = sr0;        // Salvează selecția canalului

// Extrage 'param_id' din comanda (bitii 2, 1, 0, masca 0x07)
ax0 = dm(cmd_word);
ay0= 0x0007;
ar = ax0 and ay0;
dm(param_id)= ar;      // Salvează ID-ul parametrului (fără shiftare pt. că sunt
biții cei mai puțin semnificativi)

//
=====
=====
// JUMP-URI IN FUNCTIILE DE PROCESARE (Switch/Case pe baza 'id_funcție')
//
=====
=====
ax0 = dm(id_funcție);
none = pass ax0;
ifeq jump do_agc;  // Dacă id_funcție == 0, sari la procesarea AGC

ay0 = 1;
ar = ax0 - ay0;
ifeq jump do_ale;  // Dacă id_funcție == 1, sari la procesarea ALE (Egalizare
Adaptivă de Linie)

```

```

ay0 = 2;
ar = ax0 - ay0;
ifeq jump do_mf; // Dacă id_functie == 2, sari la procesarea MF
jump do_bypass; // Dacă id_functie > 2 sau nerecunoscut, sari la funcția
care lasă semnalul neschimbat

```

```

cmd_invalida:
ax0 = 0;
dm(cmd_valida) = ax0; // Marcare comandă ca invalidă
jump do_bypass; // Sari la bypass pentru a nu distorsiona semnalul cu
procesări greșite

```

```

//
=====
=====

```

```

// FUNCTII DE PROCESARE

```

```

//
=====
=====

```

```

do_agc:
/* TODO - Secțiunea unde ar veni algoritmul Automatic Gain Control */
jump do_bypass;

```

```

do_ale:
/* TODO - Secțiunea unde ar veni algoritmul Adaptive Line Equalization */
jump do_bypass;

```

```

do_mf:
/* TODO - Secțiunea unde ar veni algoritmul de Multi-Frecvență / Filtrare */
jump do_bypass;

```

```

do_bypass:
// Funcția By-pass: la eșantionul de la intrare (x_in) și îl trimite direct la ieșire
(y_out) fără a-l modifica
ax0 = dm(x_in);

```

```
dm(y_out) = ax0;  
jump output_and_return; // Treci la scrierea rezultatului și ieșirea din  
întrerupere
```

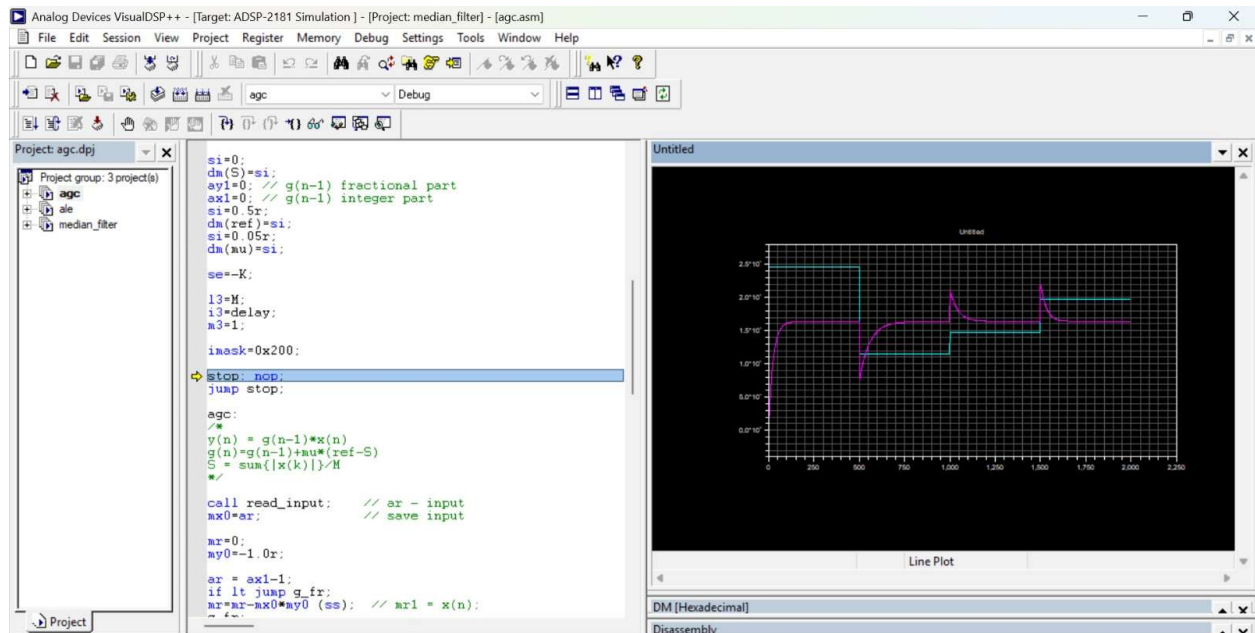
```
output_and_return:  
ax0 = dm(y_out);  
dm(tx_buf + 1) = ax0;  
dm(tx_buf + 2) = ax0;
```

```
ar = dm(PF_output);  
sr = lshift ar by 4 (hi);  
ax0 = sr1;  
dm(Prog_Flag_Data) = ax0;  
rti;
```

```
next_cmd:  
ena sec_reg;  
ax0 = dm(i3, m1);  
dm(tx_buf) = ax0;  
ax0 = i3;  
ay0 = init_cmds;  
ar = ax0- ay0;  
if gt rti;  
ax0 = 0xaf00;  
dm(tx_buf) = ax0;  
ax0 = 0;  
dm(stat_flag) = ax0;  
rti;
```

8.5. Simulări DSP (VisualDSP++):

Simulare AGC:



/* * Implementare AGC (Automatic Gain Control).

- * 1. $y(n) = g(n-1) * x(n)$ // Scalarea semnalului de intrare
- * 2. $S = \text{Sum}(|y(k)| / M)$ // Media amplitudinii pe o fereastră M
- * 3. $e(n) = \text{ref} - S$ // Eroarea fata de amplitudinea tinta
- * 4. $g(n) = g(n-1) + \mu * e(n)$ // Actualizarea castigului
- *
- * Obs: DSP-ul lucreaza cu numere sub 1 (fractionare). Castigul $g(n)$ poate fi ≥ 1 ,
- * asa ca este descompus in: $ax1$ (partea intreaga) si $ay1$ (partea fractionara).
- */

#include "def2181.h"

#define M 4 // Dimensiune fereastră delay

#define K 2 // Factor shiftare ($2^K = M$, deci shift dreapta cu 2 = impartire la 4)

```

.SECTION/DM data1;

.var/circ rx_buf[3];

.var/circ tx_buf[3];


.var/circ delay[M]; // Fereastra pentru calculul mediei

.var S; // Media semnalului (S)

.var ref; // Amplitudinea tinta (referinta)

.var mu; // Rata de adaptare (pasul mu)


//----- Tabela vectorilor de intrerupere -----

.SECTION/PM interrupts;

    jump start; rti; rti; rti; /* 00: reset */

    jump agc; rti; rti; rti; /* 04: IRQ2 - declanseaza AGC */

    rti; rti; rti; rti; /* 08: IRQL1 */

    rti; rti; rti; rti; /* 0c: IRQL0 */

    rti; rti; rti; rti; /* 10: SPORT0 tx */

    rti; rti; rti; rti; /* 14: SPORT1 rx */

    rti; rti; rti; rti; /* 18: IRQE */

    rti; rti; rti; rti; /* 1c: BDMA */

    rti; rti; rti; rti; /* 20: SPORT1 tx or IRQ1 */

    rti; rti; rti; rti; /* 24: SPORT1 rx or IRQ0 */

    rti; rti; rti; rti; /* 28: timer */

    rti; rti; rti; rti; /* 2c: power down */


.SECTION/PM seg_code;

//----- ADSP 2181 intialization -----

start:

    si=0; dm(S)=si; // Initializare medie

```

```

ay1=0;          // g(n-1) partea fractionara
ax1=0;          // g(n-1) partea intreaga

si=0.5r; dm(ref)=si;    // ref = 0.5 (Amplitudinea dorita)
si=0.05r; dm(mu)=si;    // mu = 0.05 (Pasul de adaptare a castigului)

se=-K;          // Exponent pt. shiftare (impartire la M)

l3=M; i3=delay; m3=1;   // Configurare buffer circular
imask=0x200;         // Activare intreruperi IRQ2

stop: nop;
jump stop;

//----- ISR: Automatic Gain Control -----
agc:
    call read_input;
    mx0=ar;          // mx0 = x(n)
    mr=0;
    my0=-1.0r;       // Constanta pt a realiza operatii de tip MAC cu scadere
    // --- 1. IESIRE:  $y(n) = [ax1 * x(n)] + [ay1 * x(n)]$  ---
    ar = ax1-1;
    if lt jump g_fr;   // Daca partea intreaga e 0, sari la partea fractionara
    mr=mr-mx0*my0(ss); // mr = x(n) (daca ax1 == 1)

g_fr:
    my1=ay1;
    mr=mr+mx0*my1(rnd); // mr1 = y(n) (adauga partea fractionara rotunjita)
    dm(tx_buf+2)=mr1;    // Trimite y(n) la iesire

```



```

// --- 2. PREGATIRE MEDIE:  $|y(n)| / M$  ---
ar = abs mr1;
sr=ashift ar (hi);    // sr1 =  $|y(n)| / 4$  (shiftare aritmetica cu -2)
dm(i3,m3)=sr1;        // Pune valoarea in linia de intarziere

// --- 3. MEDIE:  $S = \text{suma}(|y(k)| / M)$  ---
cntr=M-1;
mr=0, mx0=dm(i3,m3);
do sum until ce;
sum:  mr=mr-mx0*my0(ss), mx0=dm(i3,m3); // mr += element
      mr=mr-mx0*my0(rnd);    // Aduna ultimul element si rotunjeste
if mv sat mr;                // Saturare la overflow
dm(S)=mr1;                   // Salveaza noua medie S

// --- 4. EROARE:  $e(n) = \text{ref} - S$  ---
ax0=dm(ref);
ay0=dm(S);
ar=ax0-ay0;                  // ar = e(n)

// --- 5. UPDATE CASTIG:  $g(n) = g(n-1) + \mu * e(n)$  ---
my1=ar;
mx1=dm(mu);
mr=mx1*my1 (ss);             // mr1 =  $\mu * e(n)$ 

ar=mr1+ay1;                  // ay1_nou = ay1_vechi + corectie
af = pass ar;
if ge jump cont;             // Daca adunarea ramane in [0, 1), sari la salvare

```

```

// --- 6. CORECTIE OVERFLOW/UNDERFLOW CASTIG ---

// Gestioneaza trecerea limitei intre partea intreaga si cea fractionara

af = ax1-1;

if lt jump cor1;

ax1=0;          // Underflow: Castigul scade sub 1.0 (ex: 0.999)
ay1=0x7fff;     // Maxim pozitiv fractionar

rti;

cor1:

ax1=1;          // Overflow: Castigul depaseste 0.999 (devine 1.0)
ay1=0;

rti;

cont:

ay1=ar;         // Update normal (in interiorul limitelor)

rti;

//----- Rutine I/O -----

read_input:

ar=dm(rx_buf+2);

rts;

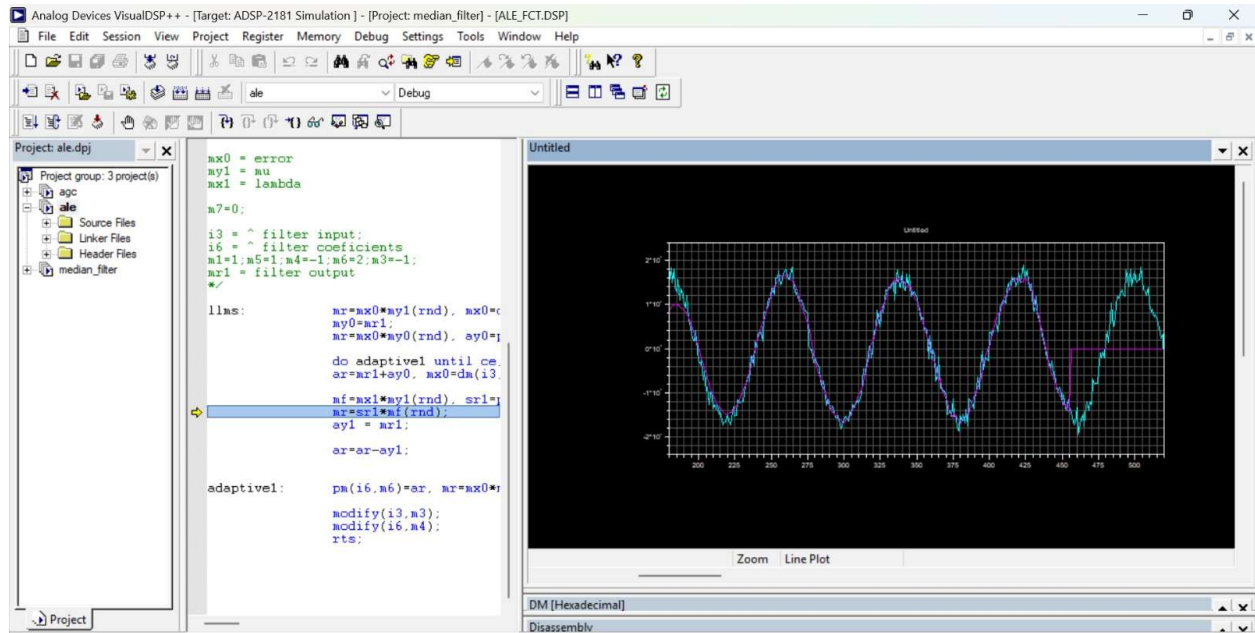
write_output:

dm(tx_buf+2)=mr1;

rts;

```

Simulare ALE:



```

/* * Implementare ALE (Adaptive Line Enhancer) cu filtru FIR
* si adaptare Leaky LMS (LLMS).
** y(n) = Sum(h_k * x(n-k))           // Iesire FIR
* e(n) = x(n) - y(n)                  // Calcul eroare
* h_k(n+1) = h_k(n)*(1 - mu*lambda) + mu*e(n)*x(n-k) // Update Leaky LMS
*/

```

```

.section/pm program;

```

```

.global fir;
.global llms;

```

```

// ----- FIR Subroutine -----

```

```

// i3 = pointer intrari x, i6 = pointer coeficienti h
// Rezultat y(n) va fi in mr1

```

```

fir:

```

```

    mr=0, mx0=dm(i3,m1), my0=pm(i6,m5);    // Init acumulator, citeste x(n), h(0)
    do sop until ce;

```

```

sop:

```

```

    mr=mr+mx0*my0(ss), mx0=dm(i3,m1), my0=pm(i6,m5); // MAC: mr += x * h

```

```

    mr=mr+mx0*my0(rnd);                    // Ultima adunare + rotunjire
    if mv sat mr;                          // Saturatie pentru a preveni overflow
    rts;

```

```

//----- LLMS Modify Subroutine -----
// mx0 = e(n), my1 = mu, mx1 = lambda, m7=0
llms:
    mr=mx0*my1(rnd), mx0=dm(i3,m1);    // mr1 = mu * e(n), incarca x(n)
    my0=mr1;                          // my0 = mu * e(n) (termen comun)

    mr=mx0*my0(rnd), ay0=pm(i6,m5);    // mr1 = x(n-k)*mu*e(n), incarca h(k)

    do adaptive1 until ce;
    ar=mr1+ay0, mx0=dm(i3,m1), ay0=pm(i6,m4); // ar = h(k) + x(n-k)*mu*e(n)

    mf=mx1*my1(rnd), sr1=pm(i6,m7);    // mf = mu * lambda
    mr=sr1*mf(rnd);                    // mr1 = h(k) * mu * lambda (termenul de leak)
    ay1 = mr1;

    ar=ar-ay1;                          // ar = update final = [update standard] - [leak]

adaptive1:
    pm(i6,m6)=ar, mr=mx0*my0(rnd);    // Salveaza noul h(k), pregateste urmatoarea inmultire

    modify(i3,m3);                      // Reset pointeri buffere
    modify(i6,m4);
    rts;

//-----
/*****
* Program testare filtru ALE (ADSP2181)
*****/

.extern fir;
.extern lms;
.extern llms;
.extern read_input;
.extern write_output;
#include "ale.h"

.section/dm data1;
.var/circ fir_d[n];    // Delay line filtru
.var/circ input[n];    // Delay line general (pentru a crea intarzierea ALE)

.section/pm data2;
.var/circ hh[n];       // Coeficienti filtru adaptiv

```

```

.section/dm data1;
.var/circ rx_buf[3];
.var/circ tx_buf[3];
.global rx_buf;
.global tx_buf;
.var save;

// Tabela vectorilor de intrerupere
.section/pm IVreset;
    jump start; rti; rti; rti;    // 00: reset
    jump input_samples;          // 04: IRQ2 (Nou esantion disponibil)
        rti; rti; rti;
    rti;    rti; rti; rti;    // 08: IRQ1
    rti;    rti; rti; rti;    // 0c: IRQ0
    rti;        rti; rti; rti; // 10: SPORT0 tx
    rti;    rti; rti; rti;    // 14: SPORT1 rx
    rti;    rti; rti; rti;    // 18: IRQE
    rti;    rti; rti; rti;    // 1c: BDMA
    rti;    rti; rti; rti;    // 20: SPORT1 tx or IRQ1
    rti;    rti; rti; rti;    // 24: SPORT1 rx or IRQ0
    rti;    rti; rti; rti;    // 28: timer
    rti;    rti; rti; rti;    // 2c: power down

.section/pm program;

start:
    // Initializari pointeri (i) si lungimi (l) pt. adresare circulara
    m1=1; m5=1; si=0;
    i2=input; l2=n;
    i4=input+delay; l4=n; // i4 ofera esantionul intarziat x(n-delay)
    i3=fir_d; l3=n;
    i6=hh; l6=n;

    // Curatare memorii date si coeficienti
    cntr=n;
    do init_buf until ce;
        dm(i2,m1)=si;
        dm(i3,m1)=si;
init_buf:
    pm(i6,m5)=si;

    imask =0x200; // Deblocare intreruperi (IRQ2)

wait:  jump wait; // Asteptare in bucla infinita semnal de la AD/DA

```

```

//----- ISR: procesare eşantion nou -----
input_samples:
    ena sec_reg;      // Foloseste registre secundare pt a proteja contextul
    ena ar_sat;       // Satureaza adunarile la limita (fara overflow)

    call read_input;   // ar = esantion nou x(n)
    dm(save)=ar;      // Salveaza pt calcul eroare

    dm(i2,m1)=ar;     // Pune x(n) in linia de intarziere principala
    ar=dm(i4,m5);     // ar = x(n-delay)
    dm(i3,m1)=ar;     // Introduce x(n-delay) in FIR

    // 1. lesirea filtrului
    cntr=n-1;
    call fir;         // lesire filtrata/prezisa ajunge in mr1 = y(n)
    call write_output; // Trimite y(n) la exterior

    // 2. Calcul eroare
    ar=dm(save);      // ar = x(n)
    ay0=ar;
    ar=ay0-mr1;       // ar = e(n) = x(n) - y(n)
    // 3. Update coeficienti
    mx0=ar;           // Setare parametri pt functia llms
    cntr=n;
    my1=mu;
    m4=-1; m6=2; m3=-1; m7=0;
    mx1=lambda;

    call llms;        // Actualizare coeficienti pe baza erorii e(n)

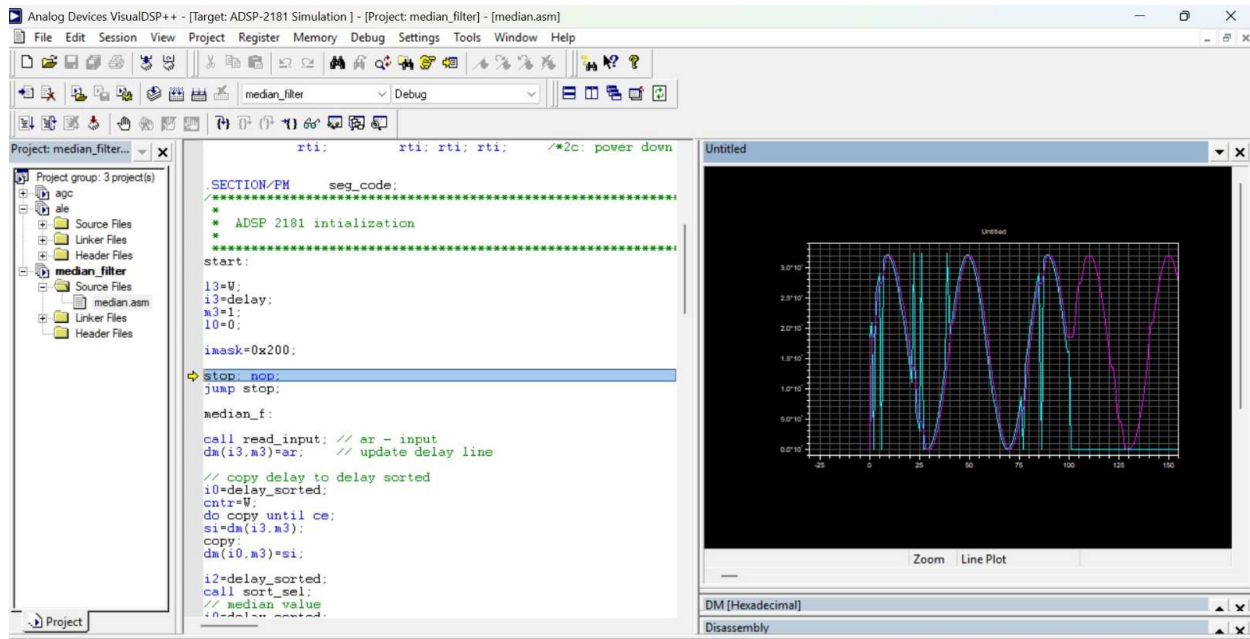
    dis sec_reg;      // Restaurare context principal
    rti;
//----- Rutine I/O SPORT -----
.section/pm program;
.global read_input;
.global write_output;
.extern rx_buf;
.extern tx_buf;

read_input:
    ar=dm(rx_buf+2);
    rts;
write_output:

```

```
dm(tx_buf+2)=mr1;
rts;
```

Simulare MF:



/* * Implementare Filtru Median cu fereastră W=3.

- * 1. La fiecare eşantion, se actualizează linia de întârziere (delay circular).
 - * 2. Se copiază datele într-un buffer temporar (delay_sorted).
 - * 3. Se sortează bufferul folosind Sortare prin Selecție (Selection Sort).
 - * 4. Se extrage valoarea din mijloc (index K=1) și se trimite la ieșire.
- */

```
#include "def2181.h"
```

```
#define W 3 // Dimensiunea ferestrei
```

```
#define K 1 // Indexul elementului median (mijlocul pentru fereastră de 3)
```

```
.SECTION/DM data1;
```

```
.var/circ rx_buf[3];
```

```
.var/circ tx_buf[3];
```

```
.var/circ delay[W]; // Buffer circular pentru istoricul semnalului
```

```
.var delay_sorted[W]; // Buffer liniar pentru sortare
```

```
/** Tabela vectorilor de intrerupere **/
```

```
.SECTION/PM interrupts;
```

```
jump start; rti; rti; rti; /* 00: reset */
```

```
jump median_f; rti; rti; /* 04: IRQ2 - esantion nou */
```

```

rti;    rti; rti; rti;    /* 08: IRQL1 */
rti;    rti; rti; rti;    /* 0c: IRQL0 */
rti;    rti; rti; rti;    /* 10: SPORT0 tx */
rti;    rti; rti; rti;    /* 14: SPORT1 rx */
rti;    rti; rti; rti;    /* 18: IRQE */
rti;    rti; rti; rti;    /* 1c: BDMA */
rti;    rti; rti; rti;    /* 20: SPORT1 tx or IRQ1 */
rti;    rti; rti; rti;    /* 24: SPORT1 rx or IRQ0 */
rti;    rti; rti; rti;    /* 28: timer */
rti;    rti; rti; rti;    /* 2c: power down */

```

.SECTION/PM seg_code;

//----- ADSP 2181 intialization -----

start:

l3=W; i3=delay; m3=1; l0=0; // Configurare buffer circular intrare

imask=0x200; // Activare intreruperi IRQ2

stop: nop;

jump stop; // Asteptare in bucla

//----- ISR: Filtru Median -----

median_f:

call read_input; // ar = intrare curenta

dm(i3,m3)=ar; // Adauga la linia de intarziere

// Copiaza datele din bufferul circular intr-unul liniar pt sortare

i0=delay_sorted;

cntr=W;

do copy until ce;

si=dm(i3,m3);

copy: dm(i0,m3)=si;

i2=delay_sorted; // i2 devine pointerul de baza pt sortare

call sort_sel;

// Extragere valoare mediana

i0=delay_sorted;

m0=K;

modify(i0,m0); // Muta pointerul la indexul K (mijloc)

ar=dm(i0,m3); // ar = valoarea mediana

dm(tx_buf+2)=ar; // Scrie valoarea la iesire

rti;


```

//----- Subrutina Sortare prin Selectie -----
// Sorteaza in loc array-ul indicat de i2.
// m0 = min, m1 = i, m2 = i+1
sort_sel:
    m3=1; l0=0; l1=0;
    cntr=W;

    // for (i = 0; i < W-1; i++)
do loop_i until ce;
    ax0=W; ay0=cntr; ar=ax0-ay0; // Calculeaza i
    m0=ar;                // m0 = min = i
    m1=ar;                // m1 = i
    ar=ar+1; m2=ar;        // m2 = i+1

    ay0=m1; ar=ax0-ay0; ar=ar-1; // Numar iteratii pt bucla j (W-i-1)
    if eq jump loop_i;      // Daca nu mai sunt elemente, sari

    cntr=ar;
    i0=i2; modify(i0,m2);    // i0 = &A[i+1]

    // for (j = i+1; j < W; j++)
do loop_j until ce;
    ax0=dm(i0,m3);          // ax0 = A[j]
    i1=i2; modify(i1,m0);
    ay0=dm(i1,m3);          // ay0 = A[min]

    ar=ax0-ay0;              // Compara A[j] cu A[min]
    if gt jump loop_j;      // Daca A[j] > A[min], continua

    ax0=W; ay0=cntr; ar=ax0-ay0; // Calculeaza j
    m0=ar;                  // min = j
loop_j: nop;

    // Swap A[i] cu A[min]
    i1=i2; modify(i1,m1); ax1=dm(i1,m3); // ax1 = tmp = A[i]
    i1=i2; modify(i1,m0); ay1=dm(i1,m3); // ay1 = A[min]

    i1=i2; modify(i1,m1); dm(i1,m3)=ay1; // A[i] = A[min]
    i1=i2; modify(i1,m0); dm(i1,m3)=ax1; // A[min] = tmp
loop_i: nop;

    rts;

```

```
//----- Rutine I/O -----
```

```
read_input:
```

```
    ar=dm(rx_buf+2);
```

```
    rts;
```

```
write_output:
```

```
    dm(tx_buf+2)=mr1; // Nota: In ISR este folosit direct dm(tx_buf+2)=ar, subrutina e optionala aici
```

```
    rts;
```

9. Simulări MATLAB:

Prin simulările MATLAB putem observa comportamentul așteptat al funcțiilor de procesare care vor fi implementate. Acestea servesc pentru validarea implementărilor viitoare pe DSP.

figura –
simulare AGC

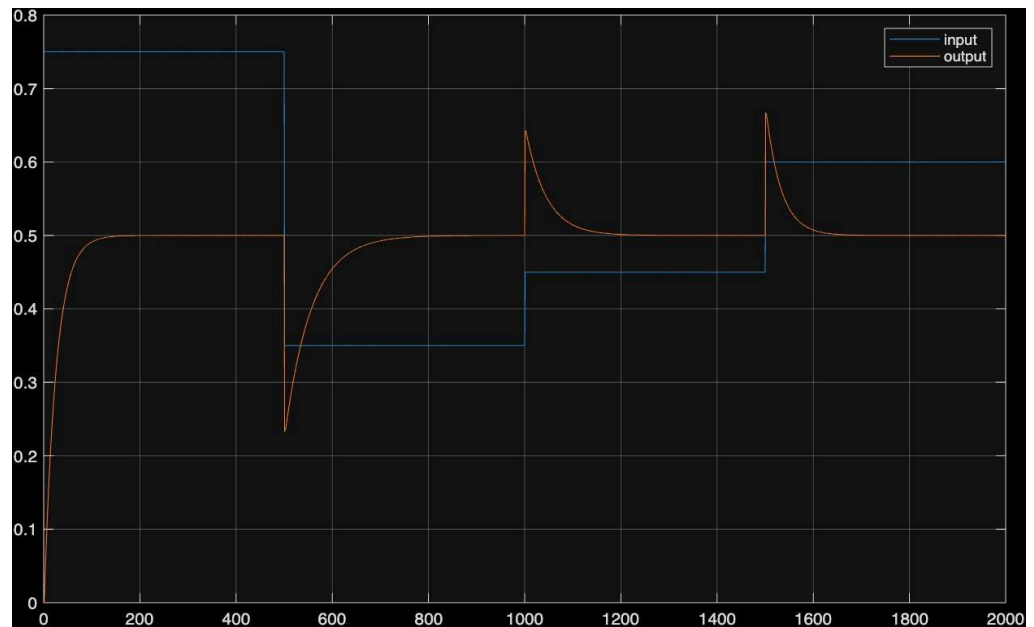


figura –
simulare ALE

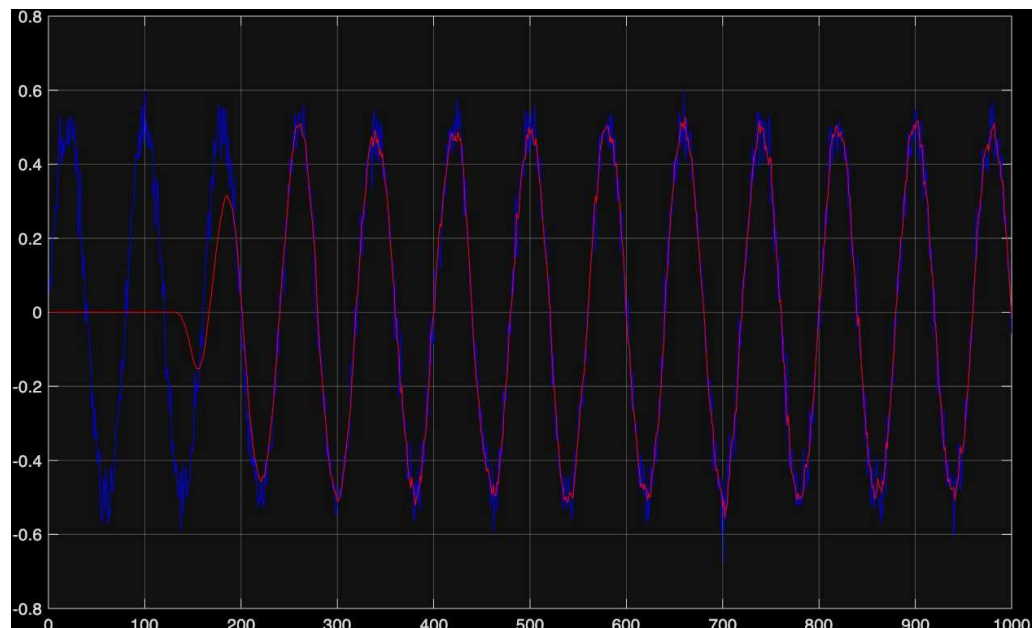


figura – simulare MF
(cu semnale separate)

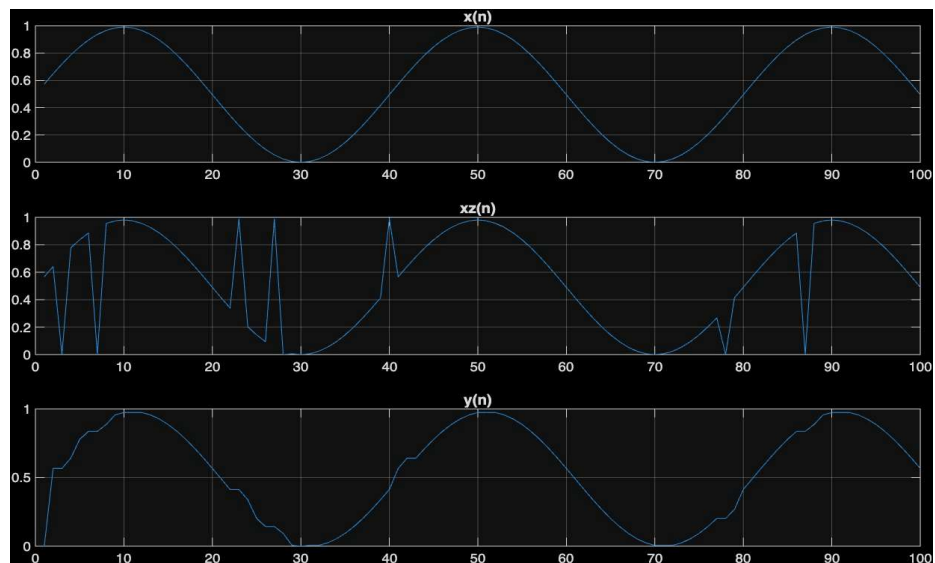
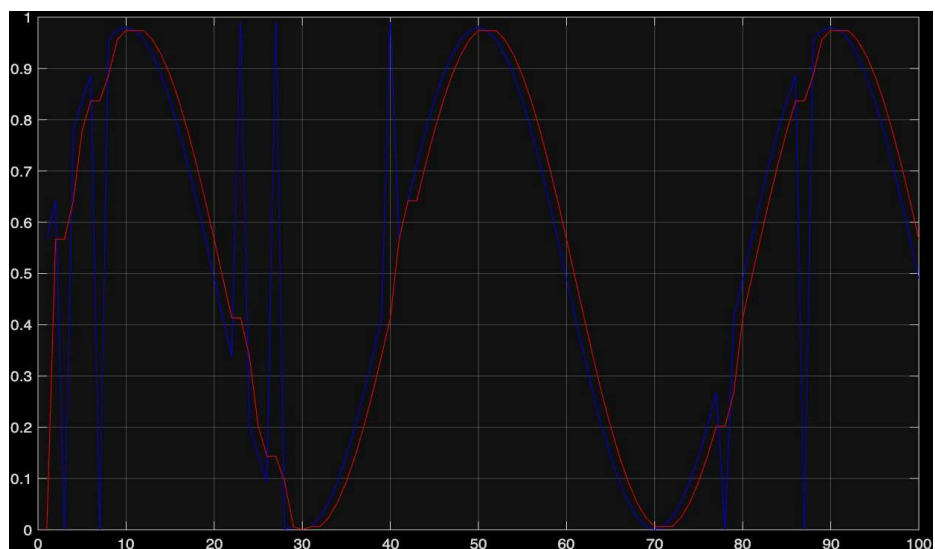


figura – simulare MF
(cu semnalele de
intrare și ieșire
suprapuse)



10.Subsistemul DSP – Finalizarea Codului, Descriere Completa

CODUL ÎN LIMBAJ DE ASAMBLARE A DSP:

```
/*
=====
=====
*
* 1. AGC (Automatic Gain Control): Ajustează dinamic câștigul pentru a
*   menține amplitudinea semnalului de ieșire la un nivel constant.
* 2. ALE (Adaptive Line Enhancer): Folosește un filtru FIR adaptiv (LMS)
*   pentru a separa un semnal util periodic de zgomotul de bandă largă.
* 3. MF (Median Filter): Elimină zgomotul impulsiv ("sare și piper") prin
*   înlocuirea eșantionului curent cu valoarea mediană a unei ferestre.
*
=====
===== */
#include "def2181.h"

// --- Definiții Constante ---
#define PORT_OUT 0xFF // Adresa portului de ieșire hardware (ex:
display LED)
#define PORT_IN 0x1FF // Adresa portului de intrare hardware (ex: DIP
switches)

#define AGC_M 4 // Lungimea ferestrei pentru media magnitudinii
(AGC)
#define AGC_K 2 // Shiftare pentru scalarea ferestrei AGC ( $2^2 = 4$ )
#define ALE_N 8 // Numărul de coeficienți (ordinul) filtrului adaptiv
ALE
#define ALE_DELAY 2 // Întârzierea aplicată pentru decorrelarea
zgomotului (ALE)
#define MF_W 3 // Lungimea ferestrei filtrului median
#define MF_K 1 // Indexul pentru extragerea valorii (ex: 1 pentru
median din 3)
```

```

// --- Declarare Buffere pentru Portul Serial (SPORT0) ---
.SECTION/DM buf_var1;
.var rx_buf[3];    // Buffer recepție (autobuffering)

.SECTION/DM buf_var2;
.var tx_buf[3] = 0xc000, 0x0000, 0x0000; // Buffer transmisie, inițializat

.SECTION/DM buf_var3;
// Comenzi de inițializare pentru codec-ul audio
.var init_cmds[13] = 0xc002, 0xc102, 0xc288, 0xc388, 0xc488, 0xc588,
                    0xc680, 0xc780, 0xc85c, 0xc909, 0xca00, 0xcc40, 0xcd00;

// --- Variabile în Memoria de Date (DM) ---
.SECTION/DM data1;

.var stat_flag;    // Flag stare inițializare
.var Flag_CDA;     // Flag prezență comandă (declanșat de interrupt)
.var cmd_word;     // Cuvânt comandă complet
.var cmd_valida;   // Indicator validitate comandă
.var id_functie;   // Funcția selectată (0=AGC, 1=ALE, 2=MF)
.var param_id;     // Parametri asociați funcției selectate
.var canal;       // Selecție canal procesat (Stânga/Dreapta)
.var x_in;        // Eșantionul curent de intrare
.var y_out;       // Eșantionul calculat pentru ieșire
.var PF_input;    // Valoare citită de pe portul de I/O
.var PF_output;   // Valoare de scris pe portul de I/O

.var sw_value;     // Valoare citită din Switch-uri (PORT_IN)
.var disp_value;   // Valoare de afișat (PORT_OUT)

// Variabile de stare și parametri pentru AGC
.var agc_delay[AGC_M]; // Istoricul valorilor absolute pentru energia
semnalului
.var agc_S;        // Suma curentă (energia estimată)

```

```

.var agc_ref;      // Referința / Nivelul dorit la ieșire
.var agc_mu;      // Rata de adaptare / Pasul de convergență
.var agc_g_int;   // Partea întreagă a câștigului calculat
.var agc_g_frac;  // Partea fracționară a câștigului calculat

// Variabile de stare și parametri pentru ALE
.var ale_input[ALE_N]; // Linia de întârziere pentru semnalul de intrare
.var ale_fir_d[ALE_N]; // Datele întârziate (decorrelate) care intră în filtru
.var ale_save;      // Salvare temporară a eșantionului curent
.var ale_mu;        // Pasul de învățare algoritmic (LMS)
.var ale_lambda;    // Factorul de 'leakage' pentru stabilitatea filtrelor

// Variabile de stare pentru Filtrul Median
.var mf_delay[MF_W]; // Fereastra circulară cu ultimele eșantioane
.var mf_delay_sorted[MF_W]; // Copia ferestrei pentru algoritmul de
sortare
.var mf_K;          // Rangul extras din fereastra sortată

// --- Coeficienți în Memoria de Program (PM) ---
.SECTION/PM pm_da;
.var ale_hh[ALE_N]; // Coeficienții adaptivi (ponderile) filtrului ALE

// --- Tabela Vectorilor de Întreruperi (IVT) ---
.SECTION/PM interrupts;
    jump start;      rti; rti; rti; // 00: Reset / Power-up
    rti;              rti; rti; rti; // 04: IRQ2
    rti;              rti; rti; rti; // 08: IRQ1
    jump isr_ia;      rti; rti; rti; // 0C: IRQ0 (Întrerupere
asincronă/Comandă)

    // 10: SPORT0 Tx (Transmisie Serială)
    ar = dm(stat_flag); // Verifică dacă mai transmitem
inițializarea
    ar = pass ar;

```

```

        if eq rti;                // Dacă e 0, codec-ul e pornit, ieşim
        jump next_cmd;            // Altfel trimitem următoarea comandă de
init

```

```

        jump input_samples; rti; rti; rti; // 14: SPORT0 Rx (Avem un eşantion
nou!)

```

```

        rti;          rti; rti; rti; // 18: IRQE
        rti;          rti; rti; rti; // 1C: BDMA
        rti;          rti; rti; rti; // 20: SPORT1 Tx
        rti;          rti; rti; rti; // 24: SPORT1 Rx
        jump isr_periodic; rti; rti; rti; // 28: Timer (Maşina de stări pentru
comenzi)
        rti;          rti; rti; rti; // 2C: Power Down

```

```

// --- SECȚIUNEA PRINCIPALĂ DE COD ---
.SECTION/PM seg_code;

```

```

start:

```

```

    // Configurare arhitectură de bază
    ax0 = b#0000100000000000;
    dm(Sys_Ctrl_Reg) = ax0;
    ena timer;                // Pornește timer-ul hardware

```

```

    // Configurare pointeri autobuffering pentru SPORT0 (Rx/Tx)
    i5 = rx_buf;
    l5 = LENGTH(rx_buf);
    i6 = tx_buf;
    l6 = LENGTH(tx_buf);
    i3 = init_cmds;
    l3 = LENGTH(init_cmds);

```

```

    m1 = 1;                    // Modificatori generici pentru pași de adrese
    m5 = 1;

```



```

// Configurare regim de funcționare interfață SPORT0
ax0 = b#0000110011010111;
dm(Sport0_Autobuf_Ctrl) = ax0;    // Activare autobuffering
ax0 = 0;
dm(Sport0_Rfsdiv) = ax0;        // Frecvențe cadre Rx/Tx
ax0 = 0;
dm(Sport0_Sclkdiv) = ax0;
ax0 = b#1000011000001111;
dm(Sport0_Ctrl_Reg) = ax0;    // Setare cuvinte de 16 biți, frame
sync

// Configurare structură buffer
ax0 = b#0000000000000011;
dm(Sport0_Tx_Words0) = ax0;
ax0 = b#0000000000000011;
dm(Sport0_Tx_Words1) = ax0;
ax0 = b#0000000000000011;
dm(Sport0_Rx_Words0) = ax0;
ax0 = b#0000000000000011;
dm(Sport0_Rx_Words1) = ax0;

// Setări de sistem adiționale
ax0 = b#0001100000000000;
dm(Sys_Ctrl_Reg) = ax0;

ifc = b#000000111111110;    // Curăță întreruperile pendinte
nop;

icntl = b#00010;            // Sensibilitate întreruperi pe front
mstat = b#1100000;        // Setare stare memorie și multiplicator

ax0 = 1;
dm(stat_flag) = ax0;        // Ridică flag-ul de inițializare activă

ena ints;                // Activează sistemul de întreruperi

```

```
imask = b#0001000001;      // Masca: Permite doar SPORT0 Tx și  
Reset deocamdată
```

```
// Declanșare trimitere prima comandă init către codec  
ax0 = dm(i6, m5);  
tx0 = ax0;
```

```
check_init:
```

```
// Buclă de așteptare până se trimit toate comenzile  
ax0 = dm(stat_flag);  
af = pass ax0;  
if ne jump check_init;
```

```
// Secvențe de verificare ACK/Status de la Codec Audio  
ay0 = 2;
```

```
check_aci1:
```

```
ax0 = dm(rx_buf);  
ar = ax0 and ay0;  
if eq jump check_aci1;
```

```
check_aci2:
```

```
ax0 = dm(rx_buf);  
ar = ax0 and ay0;  
if ne jump check_aci2;  
idle;
```

```
// Configurări extra pentru interfață post-inițializare
```

```
ay0 = 0xbf3f;  
ax0 = dm(init_cmds + 6);  
ar = ax0 and ay0;  
dm(tx_buf) = ar;  
idle;
```

```
ax0 = dm(init_cmds + 7);  
ar = ax0 and ay0;
```

```

dm(tx_buf) = ar;
idle;

ifc = b#00000011111110;      // Golește cozile de întreruperi încă o
dată
nop;

// Actualizare mască de întreruperi pentru funcționarea normală
imask = b#0000100000;

// Setări memorie de așteptare pentru comunicarea I/O
si = 0xFFFF;
dm(Dm_Wait_Reg) = si;
si = 0x0000;
dm(Prog_Flag_Comp_Sel_Ctrl) = si;

// Zeroizare variabile globale de stare și comandă
ax0 = 0;
dm(PF_input) = ax0;
dm(PF_output) = ax0;
dm(Flag_CDA) = ax0;
dm(cmd_word) = ax0;
dm(cmd_valida) = ax0;
dm(id_functie) = ax0;
dm(x_in) = ax0;
dm(y_out) = ax0;
dm(param_id) = ax0;
dm(canal) = ax0;
dm(sw_value) = ax0;
dm(disg_value) = ax0;

// --- Inițializare Parametri AGC ---
si = 0;
dm(agc_S) = si;
dm(agc_g_int) = si;

```

```

dm(agc_g_frac) = si;
si = 0.5r;           // Referință 0.5 (fracționar)
dm(agc_ref) = si;
si = 0.05r;         // Factor viteză adaptare
dm(agc_mu) = si;

// Curățare buffer întârziere AGC
i3 = agc_delay;
l3 = AGC_M;
m3 = 1;
cntr = AGC_M;
si = 0;
do clr_agc until ce;
clr_agc:
    dm(i3, m3) = si;

// --- Inițializare Parametri ALE ---
i2 = ale_input;
l2 = ALE_N;
i3 = ale_fir_d;
l3 = ALE_N;
i6 = ale_hh;
l6 = ALE_N;
cntr = ALE_N;
si = 0;
do clr_ale until ce;
dm(i2, m1) = si;      // Curățare memorie întârziere
dm(i3, m1) = si;
clr_ale:
    pm(i6, m5) = si;   // Coeficienții inițiați cu 0

si = 0.01r;
dm(ale_mu) = si;      // Pasul LMS
si = 0.001r;
dm(ale_lambda) = si;  // Coeficient de stabilitate (leakage)

```

```

// --- Inițializare Parametri MF ---
i3 = mf_delay;
l3 = MF_W;
cntr = MF_W;
si = 0;
do clr_mf until ce;
clr_mf:
    dm(i3, m3) = si;          // Zeroizare fereastră MF

    ax0 = MF_K;
    dm(mf_K) = ax0;          // Rangul dorit (medianul)

// Finalizare setup, activare întreruperi normale (timer, rx, asincron)
ena ints;
imask = b#0010100001;

// Miezul aplicației: Procesorul intră în Power Down (Idle) până la
următoarea întrerupere
main_loop:
    idle;
    jump main_loop;

// -----
// ÎNTRERUPERI ȘI MAȘINA DE STĂRI
// -----

// ISR declanșat extern - Semnalizează primirea unei comenzi
isr_ia:
    ax0 = 1;
    dm(Flag_CDA) = ax0;
    rti;

// ISR executat periodic de Timer
isr_periodic:

```

```
ena sec_reg;          // Folosim setul secundar de registre pentru  
viteză
```

```
starea_0:
```

```
    ax0 = dm(Flag_CDA);          // Verifică dacă există comandă  
asincronă nouă  
    af = pass ax0;  
    if ne jump starea_1;  
    jump starea_2;
```

```
starea_1:
```

```
    // Preluarea și citirea cuvântului de comandă din Flag-uri
```

```
    ax0 = dm(Prog_Flag_Data);
```

```
    ay0 = 0x00FF;
```

```
    ar = ax0 and ay0;
```

```
    dm(cmd_word) = ar;
```

```
    ax0 = 0;
```

```
    dm(Flag_CDA) = ax0;          // Șterge flag-ul  
rti;
```

```
starea_2:
```

```
    // Decodificare Comandă Curentă
```

```
    ax0 = dm(cmd_word);
```

```
    ay0 = 0x00C0;          // Masca pentru biții de sincronizare
```

```
    ar = ax0 and ay0;
```

```
    af = ar - ay0;
```

```
    if ne jump cmd_invalida;      // Dacă nu are antetul corect (0xC0),  
renunță
```

```
    ax0 = 1;
```

```
    dm(cmd_valida) = ax0;
```

```
    // Extragere ID Funcție (0=AGC, 1=ALE, 2=MF) - biții 4 și 5
```

```
    ax0 = dm(cmd_word);
```

```

ay0 = 0x0030;
ar = ax0 and ay0;
si = ar;
sr = lshift si by -4 (lo);
dm(id_functie) = sr0;

// Extragere ID Parametru - biții 0-2
ax0 = dm(cmd_word);
ay0 = 0x0007;
ar = ax0 and ay0;
dm(param_id) = ar;

// Extragere canal procesat (Stânga/Dreapta) - bitul 3
ax0 = dm(cmd_word);
ay0 = 0x0008;
ar = ax0 and ay0;
si = ar;
sr = lshift si by -3 (lo);
dm(canal) = sr0;

call read_extension;           // Citește starea hardware din exterior
(switch)
call apply_sw_params;          // Modifică parametrii filtrelor conform
switch-urilor
call update_display;           // Afișează starea curentă pe hardware

// Dispecer: Apelează algoritmul corect pentru eșantionul curent
ax0 = dm(id_functie);
ay0 = 0;
ar = ax0 - ay0;
if eq call do_agc;

ax0 = dm(id_functie);
ay0 = 1;
ar = ax0 - ay0;

```

```

    if eq call do_ale;

    ax0 = dm(id_functie);
    ay0 = 2;
    ar = ax0 - ay0;
    if eq call do_mf;

    rti;

cmd_invalida:
    // Bypass complet dacă avem comandă coruptă
    ax0 = 0;
    dm(cmd_valida) = ax0;
    ax0 = dm(x_in);
    dm(y_out) = ax0;
    rti;

// -----
// RUTINE AUXILIARE DE I/O ȘI DECODIFICARE (ZONA DE SWITCH-URI)
// Această secțiune gestionează interfațarea cu placa de extensie I/O.
// Extrage valorile setate fizic de utilizator pe cele 8 comutatoare (SW0-
SW7)
// și le convertește în parametri matematici pentru algoritmi.
// -----

read_extension:
    // Citirea stării fizice a pinilor
    si = IO(PORT_IN);          // Citește starea hardware a DIP switch-
urilor de la adresa I/O 0x1FF
    ax0 = si;
    ay0 = 0x00FF;              // Mască binară (0000 0000 1111 1111)
    pentru a izola doar cei 8 biți relevanți (SW0-SW7)
    ar = ax0 and ay0;
    dm(sw_value) = ar;          // Salvează configurația switch-urilor în
variabila globală 'sw_value'

```



```

    rts;

update_display:
    // Afișează pe LED-ul cu 7 segmente starea curentă a sistemului
    ax0 = dm(id_functie);      // Preia ID-ul funcției curente (0=AGC,
1=ALE, 2=MF)
    si = ax0;
    sr = lshift si by 4 (lo);    // Shiftează ID-ul funcției cu 4 poziții la
stânga (ex: 0x01 devine 0x10)
    ay0 = sr0;

    ax0 = dm(param_id);        // Preia ID-ul parametrului
    ar = ax0 or ay0;           // Combină (SAU logic) ID funcție cu ID
parametru într-un singur octet de afișare
    dm(displ_value) = ar;      // Salvează valoarea combinată

    si = ar;
    IO(PORT_OUT) = si;        // Trimite octetul spre registrul afișorului
hardware (adresa I/O 0xFF)
    rts;

apply_sw_params:
    // Funcție dispecer (de tip switch-case) care apelează rutina corectă
de decodificare
    // a switch-urilor, rutând execuția în funcție de algoritmul selectat.
    ax0 = dm(id_functie);

    ay0 = 0;
    ar = ax0 - ay0;
    if eq jump set_agc_params;  // Dacă ID == 0, sari la setarea
parametrilor specifici AGC

    ay0 = 1;
    ar = ax0 - ay0;

```

```

    if eq jump set_ale_params;    // Dacă ID == 1, sari la setarea
parametrilor specifici ALE

    ay0 = 2;
    ar = ax0 - ay0;
    if eq jump set_mf_params;    // Dacă ID == 2, sari la setarea
parametrilor specifici MF
    rts;

//
=====
=====
// DECODIFICARE PARAMETRI AGC
//
=====
=====
set_agc_params:
    // 1. Extragerea nivelului de referință (agc_ref) din primii 2 biți (SW0,
SW1)
    ax0 = dm(sw_value);
    ay0 = 0x0003;                // Mască binară 0000 0011 pentru a decupa
doar biții de pe pozițiile 0 și 1
    ar = ax0 and ay0;

    // Evaluează combinația logică formată de SW0 și SW1 (valori
posibile: 0, 1, 2 sau 3)
    ay0 = 0;
    af = ar - ay0;
    if eq jump agc_ref_025;      // Dacă valoarea e 00 binar -> setează
referința la 0.25

    ay0 = 1;
    af = ar - ay0;
    if eq jump agc_ref_050;      // Dacă valoarea e 01 binar -> setează
referința la 0.50

```

```

ay0 = 2;
af = ar - ay0;
if eq jump agc_ref_075;      // Dacă valoarea e 10 binar -> setează
referința la 0.75

    si = 0.9r; jump agc_ref_done;  // Dacă valoarea e 11 binar ->
setează referința la 0.90

agc_ref_025: si = 0.25r; jump agc_ref_done;
agc_ref_050: si = 0.5r; jump agc_ref_done;
agc_ref_075: si = 0.75r;
agc_ref_done:
    dm(agc_ref) = si;          // Salvează constanta fracționară aleasă ca
referință AGC în memorie

    // 2. Extragerea ratei de adaptare (agc_mu) din următorii 2 biți (SW2,
SW3)
    ax0 = dm(sw_value);
    si = ax0;
    sr = lshift si by -2 (lo);  // Shiftează biții la dreapta cu 2 poziții,
aducând SW2 și SW3 pe pozițiile 0 și 1
    ay0 = 0x0003;              // Aplică masca 0000 0011 peste noul
aranjament de biți
    ar = sr0 and ay0;

ay0 = 0;
af = ar - ay0;
if eq jump agc_mu_001;        // Combinația 00 -> mu = 0.01

ay0 = 1;
af = ar - ay0;
if eq jump agc_mu_005;        // Combinația 01 -> mu = 0.05

ay0 = 2;

```

```

    af = ar - ay0;
    if eq jump agc_mu_010;          // Combinația 10 -> mu = 0.1

    si = 0.2r; jump agc_mu_done;    // Combinația 11 -> mu = 0.2

agc_mu_001: si = 0.01r; jump agc_mu_done;
agc_mu_005: si = 0.05r; jump agc_mu_done;
agc_mu_010: si = 0.1r;
agc_mu_done:
    dm(agc_mu) = si;               // Salvează constanta pentru rata de
    adaptare AGC
    rts;

//
=====
=====
// DECODIFICARE PARAMETRI ALE
//
=====
=====
set_ale_params:
    // 1. Extragerea pasului de învățare algoritmic (ale_mu) din SW0,
    SW1
    ax0 = dm(sw_value);
    ay0 = 0x0003;                  // Mască pentru primii 2 biți
    ar = ax0 and ay0;

    ay0 = 0;
    af = ar - ay0;
    if eq jump ale_mu_0005;        // Setare lentă, convergență stabilă

    ay0 = 1;
    af = ar - ay0;
    if eq jump ale_mu_001;         // Setare medie

```

```

ay0 = 2;
af = ar - ay0;
if eq jump ale_mu_005;          // Setare rapidă

    si = 0.1r;  jump ale_mu_done;  // Setare foarte rapidă (crește viteza
de adaptare dar poate deveni instabil)

ale_mu_0005: si = 0.005r; jump ale_mu_done;
ale_mu_001:  si = 0.01r;  jump ale_mu_done;
ale_mu_005:  si = 0.05r;
ale_mu_done:
    dm(ale_mu) = si;          // Salvează ale_mu

    // 2. Extragerea factorului de 'leakage' (ale_lambda) din SW2, SW3
    // Previne mărirea infinită a coeficienților prin 'scurgerea' unei
fracțiuni de valoare la fiecare pas
    ax0 = dm(sw_value);
    si = ax0;
    sr = lshift si by -2 (lo);    // Shiftează SW2, SW3 pe pozițiile LSb (biții
cei mai puțin semnificativi)
    ay0 = 0x0003;
    ar = sr0 and ay0;

ay0 = 0;
af = ar - ay0;
if eq jump ale_lam_0005;

ay0 = 1;
af = ar - ay0;
if eq jump ale_lam_001;

ay0 = 2;
af = ar - ay0;
if eq jump ale_lam_005;

```

```

    si = 0.01r;  jump ale_lam_done;

ale_lam_0005: si = 0.0005r; jump ale_lam_done;
ale_lam_001: si = 0.001r;  jump ale_lam_done;
ale_lam_005: si = 0.005r;
ale_lam_done:
    dm(ale_lambda) = si;      // Salvează parametrul de stabilitate
    algoritmică
    rts;

//
=====
=====
// DECODIFICARE PARAMETRI MEDIAN FILTER (MF)
//
=====
=====
set_mf_params:
    // Extragerea rangului K (valoarea selectată din fereastra sortată)
    folosind SW0, SW1
    // Deoarece fereastra filtrului are W=3, după sortare elementele se
    află la indecșii 0, 1 și 2
    ax0 = dm(sw_value);
    ay0 = 0x0003;
    ar = ax0 and ay0;

    ay0 = 0;
    af = ar - ay0;
    if eq jump mf_k_1a;

    ay0 = 1;
    af = ar - ay0;
    if eq jump mf_k_1b;

    ay0 = 2;

```

```

    af = ar - ay0;
    if eq jump mf_k_2;

    ax0 = 3; jump mf_k_done;      // Pentru SW=11 (3), se alege indexul
3

mf_k_1a: ax0 = 1; jump mf_k_done;    // Cazul SW=00 -> Index K=1 (Se
alege MEDIANUL, elementul din mijloc)
mf_k_1b: ax0 = 1; jump mf_k_done;    // Cazul SW=01 -> Index K=1
(Același comportament de filtru median)
mf_k_2: ax0 = 2;                  // Cazul SW=10 -> Index K=2 (Se alege
MAXIMUL. Transformă algoritmul în filtru trece-sus ne-linar de tip MAX)
mf_k_done:
    dm(mf_K) = ax0;              // Salvează indexul utilizat ulterior pentru
    extracția din array-ul sortat
    rts;

// -----
// IMPLEMENTARE ALGORITMI DSP PENTRU EȘANTIONUL CURENT (x_in
-> y_out)
// Această secțiune conține logica matematică a fiecărui filtru în parte.
// -----

do_agc:
    // === PASUL 1: CONFIGURARE POINTERI ȘI SCALARE ===
    // Configurare pointer pentru a naviga prin istoricul eșantioanelor
    i3 = agc_delay;
    l3 = AGC_M;                  // Lungimea bufferului circular (ex. 4)
    m3 = 1;                      // Pasul de incrementare al pointerului

    // Exponentul pentru scalare: 'se' controlează shiftarea aritmetică.
    // Valoarea negativă (-AGC_K) este echivalentă cu o împărțire la
    2^AGC_K (ex: împărțire la 4).
    se = -AGC_K;

```

```

ar = dm(x_in);
mx0 = ar;           // Încărcăm x_in în registrul multiplicator mx0

// === PASUL 2: APLICARE CÂȘTIG DINAMIC (g * x_in) ===
// Deoarece DSP-ul cu virgulă fixă nu suportă valori > 0.999 direct
// într-un registru fracționar,
// câștigul (g) este împărțit în parte întreagă (agc_g_int) și parte
// fracționară (agc_g_frac).
mr = 0;             // Curățăm acumulatorul
my0 = -1.0r;        // Constantă ajutătoare pentru artificii de
calcul
ax1 = dm(agc_g_int);
ay1 = dm(agc_g_frac);

// Verificăm dacă partea întreagă a câștigului este >= 1
ar = ax1 - 1;
if lt jump agc_g_fr; // Dacă partea întreagă e < 1 (adică 0),
sărim la înmulțirea fracționară

// Dacă partea întreagă este >= 1, adunăm x_in (din mx0) la
acumulare
mr = mr - mx0 * my0 (ss); // Operație de tip MAC care practic
face mr = mr + x_in
agc_g_fr:
my1 = ay1;           // Încărcăm partea fracționară a câștigului
mr = mr + mx0 * my1 (rnd); // Adunăm produsul (x_in *
agc_g_frac) cu rotunjire (rnd)

// Rezultatul scalat este gata
dm(y_out) = mr1;     // Salvăm eșantionul final rezultat

// === PASUL 3: CALCULUL ENERGIEI (Mediei Modulelor) ===
ar = abs mr1;        // Luăm valoarea absolută a ieșirii curențe
|y_out|

```



```

    sr = ashift ar (hi);          // Scalăm |y_out| (împărțim la M) prin
shiftarea dictată de 'se'
    dm(i3, m3) = sr1;           // Suprascrim cel mai vechi eșantion din
istoricul circular cu cel nou

```

```

    // Însumăm toate valorile scalate din istoricul circular pentru a
obține media (S)
    cntr = AGC_M - 1;
    mr = 0, mx0 = dm(i3, m3);    // Preîncărcăm primul element
    do agc_sum until ce;         // Buclă pentru adunarea eșantioanelor
agc_sum:

```

```

    mr = mr - mx0 * my0 (ss), mx0 = dm(i3, m3); // Acumulare progresivă

    // Finalizarea buclei
    mr = mr - mx0 * my0 (rnd);
    if mv sat mr;                // Dacă există depășire (overflow), saturează
la valoarea maximă
    dm(agc_S) = mr1;             // Salvăm energia semnalului (S)
calculată

```

```

// === PASUL 4: ACTUALIZAREA CÂȘTIGULUI PENTRU VIITOR ===

```

```

// Formula: Eroare = Referință - S

```

```

ax0 = dm(agc_ref);

```

```

ay0 = dm(agc_S);

```

```

ar = ax0 - ay0;                // ar conține eroarea

```

```

// Înmulțim eroarea cu rata de adaptare mu: delta_G = mu * eroare

```

```

my1 = ar;

```

```

mx1 = dm(agc_mu);

```

```

mr = mx1 * my1 (ss);

```

```

// Actualizăm câștigul: G_nou = G_vechi_fracționar + delta_G

```

```

ar = mr1 + ay1;

```

```

// === GESTIUNEA TRECERII PESTE 1 A CÂȘTIGULUI ===

```

```

    // Verificăm dacă prin adăugarea deltei, partea fracționară "dă pe
    afară" (depășește 1.0 sau scade sub 0)
    af = pass ar;
    if ge jump agc_cont;          // Dacă adunarea e validă și pozitivă,
    sărim la final

    // Dacă am scăzut sub zero (underflow din partea fracționară)
    af = ax1 - 1;                // Verificăm dacă aveam parte întreagă
    if lt jump agc_cor1;          // Dacă nu aveam parte întreagă și am
    scăzut sub 0, facem underflow absolut

    ax1 = 0;                     // Dacă aveam 1 întreg, acum a devenit 0.999...
    ay1 = 0x7fff;                // Setăm partea fracționară la maximul pozitiv
    (aprox. 0.999)
    jump agc_save;
agc_cor1:
    ax1 = 1;                     // Am depășit limita de sus (overflow). Devenim
    1.0.
    ay1 = 0;                     // Resetăm partea fracționară la 0
    jump agc_save;
agc_cont:
    ay1 = ar;                     // Actualizare normală, fără a traversa bariera
    de 1.0

agc_save:
    dm(agc_g_int) = ax1;          // Salvăm înapoi partea întreagă
    dm(agc_g_frac) = ay1;        // Salvăm înapoi partea fracționară
    rts;

do_ale:
    // Filtru FIR adaptiv cu algoritm LMS (Least Mean Squares)
    ena ar_sat;                  // Activează algebra cu saturație pentru
    unitatea ALU

    m1 = 1;                      // Pas circular 1

```

```

m5 = 1;                // Pas circular 1

// === PASUL 1: ACTUALIZARE LINII DE ÎNTÂRZIERE ===
i2 = ale_input;        // Pointer la intrarea curentă a liniei
principale
l2 = ALE_N;

i4 = ale_input + ALE_DELAY;    // Pointer pentru extragerea
eșantionului decorrelat
// (zgomotul e aleator, deci se decorrelează rapid;
semnalul util rămâne)
l4 = ALE_N;

i3 = ale_fir_d;        // Linia de intrare în filtrul FIR efectiv
l3 = ALE_N;

i6 = ale_hh;          // Pointer către vectorul cu coeficienții filtrului
(ponderile W)
l6 = ALE_N;

ar = dm(x_in);
dm(ale_save) = ar;    // Salvăm eșantionul curat pentru a-l
folosi la calculul erorii

// Introducem eșantionul curent în prima linie de întârziere
dm(i2, m1) = ar;

// Extragem eșantionul "vechi" (întârziat cu ALE_DELAY) și îl punem în
a doua linie, de intrare în FIR
ar = dm(i4, m5);
dm(i3, m1) = ar;

// === PASUL 2: APLICARE FILTRU FIR ===
cntr = ALE_N - 1;    // Setăm contorul pentru bucla de Multiply-
Accumulate

```

```
    call ale_fir;          // mr1 va conține produsul scalar (ieșirea y_out  
estimată)
```

```
    dm(y_out) = mr1;      // Salvăm ieșirea
```

```
    // === PASUL 3: CALCUL EROARE PREDICȚIE ===
```

```
    // Formula:  $e(n) = \text{Semnal\_Original} - \text{Semnal\_Estimat\_de\_Filtru}$ 
```

```
    ar = dm(ale_save);    // Aducem înapoi x_in-ul curat
```

```
    ay0 = ar;
```

```
    ar = ay0 - mr1;      // ar = x_in - y_out
```

```
    mx0 = ar;            // mx0 = e(n) (Încărcăm eroarea în  
multiplicator)
```

```
    // === PASUL 4: ACTUALIZARE COEFICIENȚI (LEAKY LMS) ===
```

```
    cntr = ALE_N;
```

```
    my1 = dm(ale_mu);    // Încărcăm pasul de adaptare (mu)
```

```
    mx1 = dm(ale_lambda); // Încărcăm factorul de scurgere /  
uitare (lambda)
```

```
    // Modificatori specifici pentru a naviga eficient în paralel prin  
coeficienți și memorie de date
```

```
    m4 = -1;
```

```
    m6 = 2;
```

```
    m3 = -1;
```

```
    m7 = 0;
```

```
    call ale_llms;      // Apelăm rutina de modificare a  
coeficienților  
    rts;
```

```
ale_fir:
```

```
    // Buclă de Produs și Acumulare (MAC) optimizată pentru procesor  
de semnal
```

```
    // Extrage simultan o dată din memorie și un coeficient din Program  
Memory (PM)
```

```
    mr = 0, mx0 = dm(i3, m1), my0 = pm(i6, m5);
```

```

do ale_sop until ce;
ale_sop:
    // Acumulează:  $mr = mr + Data * Coeficient$ 
     $mr = mr + mx0 * my0$  (ss),  $mx0 = dm(i3, m1)$ ,  $my0 = pm(i6, m5)$ ;

    // Rotația finală cu rotunjire pentru ultimul element
     $mr = mr + mx0 * my0$  (rnd);
    if mv sat mr;           // Saturează dacă suma finală a depășit
capacitatea (prevenire zgomot digital)
    rts;

ale_llms:
    // Buclă super-optimizată la nivel de ciclu mașină pentru LMS
    // Modifică ponderile:  $h(n+1) = h(n) + \mu * e(n) * x(n-k) -$ 
(leakage_factor *  $h(n)$ )
     $mr = mx0 * my1$  (rnd),  $mx0 = dm(i3, m1)$ ; //  $mr1 = \mu * eroare$ .
Extrage un eșantion vechi
     $my0 = mr1$ ;           // Păstrează termenul ( $\mu * eroare$ ) în
my0

     $mr = mx0 * my0$  (rnd),  $ay0 = pm(i6, m5)$ ; //  $mr1 = (\mu * eroare) * x(n-$ 
k). Extrage ponderea curentă  $h(k)$ 

do ale_adapt until ce;
     $ar = mr1 + ay0$ ,  $mx0 = dm(i3, m1)$ ,  $ay0 = pm(i6, m4)$ ; //  $ar = h(k) +$ 
delta. Pregătește următoarele date

     $mf = mx1 * my1$  (rnd),  $sr1 = pm(i6, m7)$ ; // Calculăm factorul de
"Leakage" în feedback-ul multiplicatorului
     $mr = sr1 * mf$  (rnd);           // Termenul care trage ponderile ușor
spre 0 pentru stabilitate
     $ay1 = mr1$ ;
     $ar = ar - ay1$ ;           // Scădem leakage-ul din noua pondere
ale_adapt:
    // Salvăm noua pondere înapoi în Program Memory la poziția ei

```

```

pm(i6, m6) = ar, mr = mx0 * my0 (rnd);

modify(i3, m3);           // Reajustare pointeri la finalul buclei
modify(i6, m4);
rts;

do_mf:
    // Filtrul Median: Înlocuiește eșantionul cu valoarea de mijloc din
    fereastră pentru a elimina zgomotul impulsiv
    m1 = 1;
    m3 = 1;

    // === PASUL 1: ACTUALIZARE FEREASTRĂ ===
    i3 = mf_delay;
    l3 = MF_W;             // Fereastră circulară de mărime 3
    ar = dm(x_in);
    dm(i3, m3) = ar;       // Inserăm noul eșantion înlocuindu-l pe cel
    mai vechi

    // === PASUL 2: COPIERE PENTRU SORTARE ===
    // Sortarea modifică ordinea, așa că trebuie să lucrăm pe o copie
    pentru a nu strica memoria istorică
    i0 = mf_delay_sorted;  // Array-ul destinație (liniar, l0=0)
    l0 = 0;
    cntr = MF_W;
    do mf_copy until ce;
    si = dm(i3, m3);
mf_copy:
    dm(i0, m3) = si;       // Copiere element cu element

    // === PASUL 3: SORTARE ===
    i2 = mf_delay_sorted;  // Transmitem adresa de bază a array-
    ului copiat
    call mf_sort_sel;       // Sortăm crescător prin Selection Sort

```

```

// === PASUL 4: EXTRAGERE VALOARE MEDIANĂ ===
i0 = mf_delay_sorted;
l0 = 0;
ax0 = dm(mf_K);          // mf_K reprezintă indexul dorit (ex. 1
pentru elementul de mijloc dintr-un array de 3)
m0 = ax0;
modify(i0, m0);          // Mutăm pointerul `i0` direct la indexul
`K`
ar = dm(i0, m3);          // Citim valoarea
dm(y_out) = ar;          // O trimitem la ieșire ca fiind rezultatul
"curățat"
rts;

```

```

mf_sort_sel:
    // Algoritm de sortare in-place: Selection Sort
    // Deși complexitatea este  $O(N^2)$ , este foarte rapid pentru N mic
    (N=3) și necesită puține variabile

```

```

    m3 = 1;

```

```

    l0 = 0;

```

```

    l1 = 0;

```

```

    cntr = MF_W;          // Bucla exterioară (i)

```

```

    do mf_loop_i until ce;

```

```

    // Calculăm indexul curent `i`

```

```

    ax0 = MF_W;

```

```

    ay0 = cntr;

```

```

    ar = ax0 - ay0;

```

```

    m0 = ar;              // m0 = indexul minimului curent presupus

```

```

    m1 = ar;              // m1 = index i

```

```

    ar = ar + 1;

```

```

    m2 = ar;              // m2 = index i + 1 (start pt bucla interioară)

```

```

    // Verificăm câte elemente mai avem de sortat în bucla interioară

```

```

    ay0 = m1;

```

```

ar = ax0 - ay0;
ar = ar - 1;

if eq jump mf_skip_inner;      // Dacă am ajuns la ultimul element,
terminăm

cntr = ar;
i0 = i2;
modify(i0, m2);                // Setăm pointerul de căutare a minimului
la `i+1`

// Bucla interioară (j) pentru aflarea elementului minim din restul
subvectorului
do mf_loop_j until ce;
ax0 = dm(i0, m3);              // Citim elementul de la indexul `j`
i1 = i2;
modify(i1, m0);
ay0 = dm(i1, m3);              // Citim elementul minim curent

ar = ax0 - ay0;
if gt jump mf_skip_min;        // Dacă elementul(j) > minim, nu ne
interesează, sărim

// Am găsit un nou minim, actualizăm indexul minimului în m0
ax0 = MF_W;
ay0 = cntr;
ar = ax0 - ay0;
m0 = ar;
mf_skip_min:
mf_loop_j:
nop;

// === PAS DE INTERSCHIMBARE (SWAP) ===
// Acum că am găsit minimul absolut din subvector, îl interschimbăm
cu elementul de la poziția `i`

```



```

    i1 = i2;
    modify(i1, m1);
    ax1 = dm(i1, m3);           // ax1 = array[i]

    i1 = i2;
    modify(i1, m0);
    ay1 = dm(i1, m3);           // ay1 = array[min]

    i1 = i2;
    modify(i1, m1);
    dm(i1, m3) = ay1;           // array[i] = ay1 (minim)

    i1 = i2;
    modify(i1, m0);
    dm(i1, m3) = ax1;           // array[min] = ax1 (valoarea veche)

mf_skip_inner:
    nop;
mf_loop_i:
    nop;

    m1 = 1;
    rts;

// -----
// ÎNTRERUPERI SPORT PENTRU COMUNICARE AUDIO SERIALĂ
// Se ocupă cu recepția datelor de la ADC-ul codec-ului și transmisia
// înapoi spre DAC
// -----

input_samples:
    // A fost recepționat un pachet (cadru) complet pe portul serial
SPORT0
    ena sec_reg;                // Răspuns rapid ISR prin schimbarea setului
    activ de registre

```

```

// Preluăm eşantionul din rx_buf.
// rx_buf[2] şi rx_buf[1] reprezintă canalele audio stânga/dreapta
recepţionate.
    sr1 = dm(rx_buf + 2);
    dm(x_in) = sr1;          // Alimentăm logica DSP de mai sus punând
data în x_in

    // Reîmprospătare stare hardware adiţională (citim portul IN, scriem
direct la OUT)
    si = IO(PORT_IN);
    IO(PORT_OUT) = si;

    // === RUTARE CANALE AUDIO ===
    // Algoritmii prelucrează un singur canal (mono). Verificăm canalul
activ.
    ax0 = dm(canal);
    ay0 = 0;
    ar = ax0 - ay0;
    if eq jump out_left;

    // Cazul 1: Procesăm canalul dreapta
    ax0 = dm(y_out);        // Extragem rezultatul procesat din y_out
    dm(tx_buf + 2) = ax0;    // Îl plasăm pe canalul dreapta de
transmisie

    sr1 = dm(rx_buf + 1);    // Luăm canalul stânga brut, de la
intrare...
    dm(tx_buf + 1) = sr1;    // ...şi îl trecem neatins (BYPASS) către
ieşire
    jump out_done;

out_left:
    // Cazul 2: Procesăm canalul stânga (simetric cu logica de mai sus)
    ax0 = dm(y_out);

```

```

    dm(tx_buf + 1) = ax0;          // Rezultatul procesat pe stânga

    sr1 = dm(rx_buf + 2);
    dm(tx_buf + 2) = sr1;          // Dreapta primește bypass brut
out_done:

    // Salvare comandă asincronă detectată la nivelul codec-ului (Dacă
    există ceva adițional pe magistrala de stare)
    ax0 = dm(Prog_Flag_Data);
    ay0 = 0x00FF;
    ar = ax0 and ay0;
    dm(PF_input) = ar;

    rti;                          // Return din întrerupere. Hardware-ul trimite
    buffer-ul serial (tx_buf).

next_cmd:
    // ISR de transmisie serială declanșat doar în faza de
    BOOT/INIȚIALIZARE
    // Trimite succesiv (secvențial) array-ul init_cmds către codec-ul
    hardware AD1847
    ena sec_reg;

    ax0 = dm(i3, m1);             // Preia următoarea comandă de setare
    (ex. sample rate, gain codec, muting)
    dm(tx_buf) = ax0;             // O plasează în buffer-ul de trimitere pe
    serial

    ax0 = i3;                    // Verificăm adresa curentă a pointerului i3
    ay0 = init_cmds;
    ar = ax0 - ay0;              // Verificăm dacă mai avem elemente în array
    (i3 > baza array-ului?)
    if gt rti;                   // Dacă da, ieșim și așteptăm următoarea
    întrerupere pentru a trimite următorul cuvânt

```

```

// === FINAL DE INIȚIALIZARE ===
// Am ajuns la capătul array-ului de comenzi.
ax0 = 0xaf00;           // Cuvânt generic de umplură / status
neutru pentru a încheia
dm(tx_buf) = ax0;

ax0 = 0;
dm(stat_flag) = ax0;    // Oprim logica de inițializare. De acum
încolo, întreruperile tx vor fi ignorate.
rti;

```

Descrierea Completa a Functionalitatii Codului:

10.1. Integrarea Hardware: Gestiunea Plăcii de Extensie I/O

O cerință fundamentală a sistemului este capacitatea de a modifica parametrii de filtrare fără a fi necesară reprogramarea DSP-ului. Această facilitate este oferită de placa de extensie I/O, descrisă în schemele electrice ale proiectului. Placa include un bloc de comutatoare (DIP switches) pentru introducerea datelor și un afișor cu 7 segmente pentru monitorizare.

Inițializarea stărilor I/O: Un pas premergător fluxului de execuție I/O este faza de setare a stărilor inițiale ale sistemului. În cadrul secțiunii de zeroizare a variabilelor globale de stare și comandă din codul de inițializare, variabilele specifice plăcii de extensie, respectiv `sw_value` (pentru citirea portului de intrare) și `disp_value` (pentru afișare), sunt inițializate explicit cu valoarea 0 în memoria de date. Această resetare a memoriei are loc înainte de finalizarea configurărilor sistemului de întreruperi și înainte ca procesorul să intre în bucla infinită de așteptare (`main_loop`), asigurând un mediu curat înainte de prima execuție a subrutinei `read_extension`.

Pentru a integra această componentă hardware în fluxul software, au fost definite spații de memorie I/O specifice, mapate fizic pe magistrala de adrese a DSP-ului:

- **PORT_IN (0x1FF):** Adresa de memorie asociată bufferului octal Tri-state (74LS244) de pe placa de extensie. Citirea de la această adresă aduce în acumulatorul DSP-ului starea fizică a comutatoarelor (High/Low).
- **PORT_OUT (0xFF):** Adresa asociată registrului cu latch (74LS374). Scrierea la această adresă trimite date către afișorul LED (LSD030BUE) pentru a indica starea internă.

Fluxul de execuție I/O: În cadrul mașinii de stări, subrutina `read_extension` interoghează **PORT_IN**, aplică o mască de 8 biți (0x00FF) și salvează valoarea în variabila `sw_value`. Imediat după, `apply_sw_params` decodifică această valoare pentru a suprascrie parametrul algoritmulor:

- Pentru AGC: Selectează nivelul de referință dorit (`agc_ref` între 0.25 și 0.9) și viteza de adaptare (`agc_mu`).
- Pentru ALE: Ajustează pasul de învățare algoritmic (`ale_mu`) și factorul de stabilitate (`ale_lambda`).
- Pentru MF: Modifică rangul extras din fereastra de sortare (`mf_K`), permițând selectarea valorii minime, mediane sau maxime.

În final, `update_display` compune un octet (combinând ID-ul funcției și ID-ul parametrului) pe care îl scrie la adresa **PORT_OUT**, validând vizual comanda primită.

10.2. Organizarea Memoriei (Arhitectura Harvard)

Pentru a garanta execuția instrucțiunilor într-un singur ciclu-mașină, codul exploatează arhitectura Harvard a procesorului ADSP-2181, care dispune de magistrale separate pentru date (DM) și program (PM).

- **Gestionarea I/O Audio (Autobuffering):** Pentru a nu încărca procesorul cu rutine de recepție individuale, interfața serială SPORT0 este configurată în modul de autobuffering. Bufferele `rx_buf[3]` și `tx_buf[3]` sunt declarate circulare (`.var/circ`). Hardware-ul DMA intern al DSP-ului scrie automat eșantioanele provenite de la CODEC-ul AD1847 în `rx_buf` și declanșează o întrerupere (`input_samples`) doar când întregul cadru multicanal a fost recepționat.
- **Separarea Datelor Algoritmice:** Memoria de Date (`.SECTION/DM data1`) conține exclusiv variabilele de stare, liniile de întârziere (ex. `agc_delay`,

ale_input) și bufferele de sortare (mf_delay_sorted). În contrast, coeficienții adaptivi ai filtrului ALE (ale_hh) sunt stocați în Memoria de Program (.SECTION/PM pm_da). Această plasare strategică permite unității MAC (Multiply-Accumulate) să extragă simultan un eșantion de date (din DM) și un coeficient (din PM) în exact același ciclu de ceas.

10.3. Mașina de Stări (State Machine) și Sincronizarea

Logica de control dictată de microcontrolerul ARM este interpretată de DSP printr-o mașină de stări declanșată periodic de Timer-ul intern (isr_periodic). Comunicarea este inițiată asincron prin semnalul conectat la pinul de întrerupere hardware (care declanșează isr_ia), setând variabila globală Flag_CDA.

Tranzițiile Mașinii de Stări:

1. Starea 0: Evaluează Flag_CDA. Dacă există o cerere de la ARM, avansează la citire; altfel, sare la procesarea normală.
2. Starea 1: Citește portul fizic Prog_Flag_Data (unde ARM-ul a expus pachetul de 8 biți) și îl izolează în cmd_word.
3. Starea 2 (Validare și Rutare): Execută un test de integritate verificând biții de sincronizare (masca 0xC0). Odată validată, extrage prin operații de shiftare la dreapta (lshift) codul funcției, canalul audio vizat și parametrul aferent. Printr-o structură decizională de tip switch-case (implementată prin instrucțiuni succesive ar = ax0 - ay0; if eq jump ...), execuția este direcționată către algoritmul selectat (do_agc, do_ale, do_mf).

10.4. Analiza Implementării Algoritmilor DSP

Fiecare rutină de procesare manipulează eșantionul curent x_in pentru a produce y_out, folosind setul de instrucțiuni optimizat matematic al ADSP-2181.

10.4.1. Automatic Gain Control (AGC)

Funcția stabilizează amplitudinea semnalului de ieșire. Modulul calculează energia medie $S = M1 \sum_{k=0}^{M-1} |y(n-k)|$ prin medierea modulelor stocate în fereastra `agc_delay`. Câștigul dinamic este recalculat iterativ utilizând formula integratorului discret: $g(n) = g(n-1) + \mu[REF - S]$.

Deoarece procesoarele DSP pe 16 biți cu virgulă fixă nu pot reprezenta nativ numere supraunitare mari (formatul standard este fractional 1.15, intervalul $[-1, 0.999]$), câștigul a fost divizat în două variabile: `agc_g_int` (partea întreagă) și `agc_g_frac` (partea fracționară). Codul conține o rutină complexă (`cor1`) care supraveghează flag-ul de depășire aritmetică (`af = pass ar`) pentru a propaga corect unitățile de la partea fracționară la cea întreagă.

10.4.2. Adaptive Line Enhancer (ALE)

Separă semnalul util periodic de zgomot prin implementarea unui filtru FIR adaptiv. Pentru filtrare (`ale_fir`), procesorul folosește bucla ultra-optimizată de Produs și Acumulare (`mr = mr + mx0 * my0 (ss)`), procesând vectorii cu pointeri cu autoincrementare circulară. Pentru adaptarea coeficienților (`ale_llms`), se folosește un algoritm LMS (Least Mean Squares) modificat pentru a include un factor de scurgere (Leakage) λ , prevenind divergența ecuației la intrări instabile: $h(k) = (1 - \mu \cdot \lambda)h(k) + \mu \cdot x(n-k-D) \cdot e(n)$.

Unitatea ALU și Multiplicatorul sunt folosite în tandem. Eroarea $e(n)$ este calculată ca diferență între eșantionul întârziat și predicția filtrului. Instrucțiunea `mf = mx1 * my1 (rnd)` stochează factorul de scurgere în registrul de feedback intern al multiplicatorului, aplicându-l instantaneu peste noile ponderi calculate.

10.4.3. Median Filter (MF)

Proiectat pentru a elimina zgomotul de tip impuls , filtrul utilizează o tehnică de analiză a rangului pe o fereastră de dimensiune $W=3$. La fiecare eșantion, fereastra circulară este protejată prin copierea într-un buffer temporar de lucru (mf_delay_sorted).

Deoarece procesoarele DSP nu dispun de instrucțiuni de sortare nativă, a fost implementat algoritmul Selection Sort (Sortare prin Selecție) la nivel de instrucțiune (sort_sel). Cele două bucle imbricate verifică și interschimbă direct valorile în memorie, minimizând utilizarea registrelor interne de date. Eșantionul de ieșire $y(n)$ este apoi preluat de la indexul dorit (K), oferind instantaneu valoarea mediană care respinge vârfurile anormale ale semnalului de intrare.